

Matrix Multiplication Accelerator Design

Aman Arora



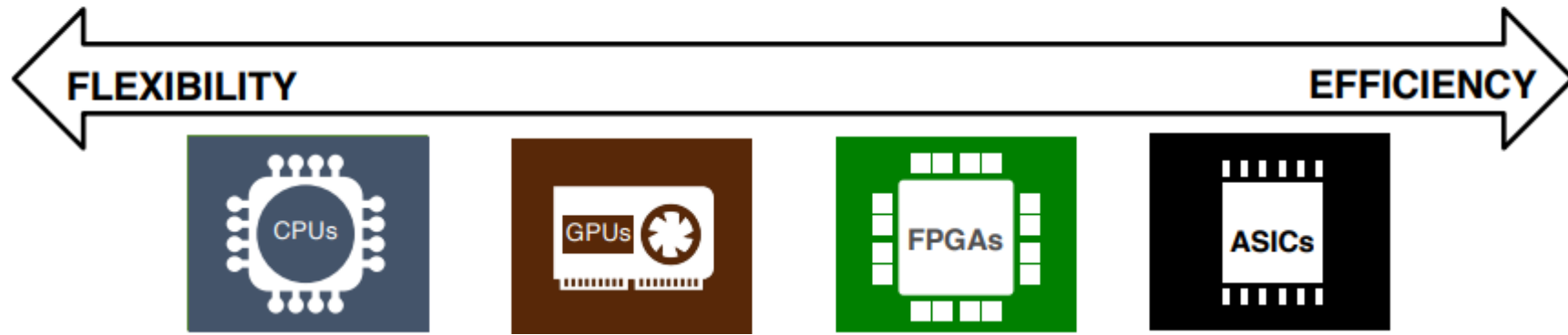
Basics

- What does it mean to compute in software?
 - Using a “general purpose” CPU
- What does it mean to compute in hardware?
 - Not using a CPU
 - Using specialized hardware instead

What is hardware acceleration?

- Offload certain tasks to 'hardware' to 'accelerate' them
- Why?
 - Run more efficiently
 - Lower power
 - Lower time
 - Lower energy consumption

Flexibility – Efficiency Tradeoff



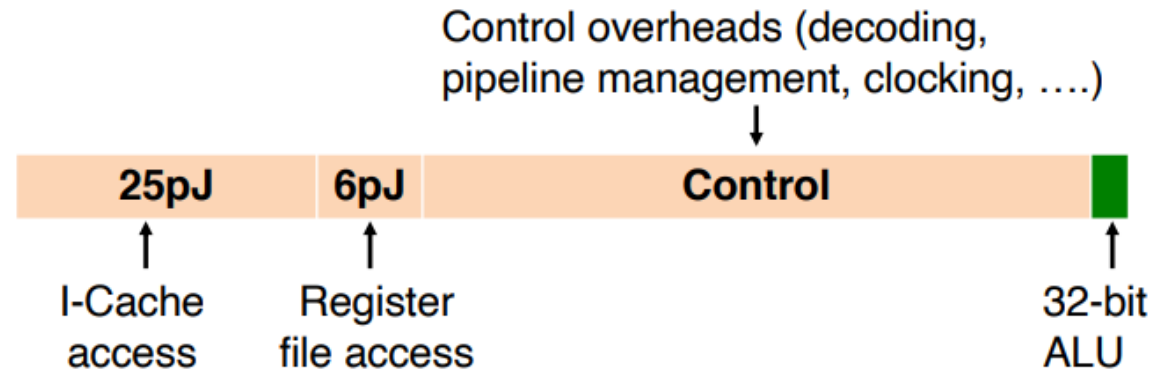
But why is the CPU less energy efficient?

Energy breakdown for an instruction

Rough energy costs for various CPU operations (45nm at 0.9V)

Integer	
Add	
8 bit	0.03pJ
32 bit	0.1pJ
Mult	
8 bit	0.2pJ
32 bit	3.1pJ

Memory	
Cache	(64bit)
8KB	10pJ
32KB	20pJ
1MB	100pJ
DRAM	1.3-2.6nJ



[Source] M. Horowitz, Computing's energy problem (and what we can do about it), ISSCC'2014.

Main sources of acceleration

- Reduced overhead
 - Fetch, decode, etc.
- Specialization
 - Domain specific data types
- Parallelism
 - Many processing elements
- Optimized memory
- Optimized communication

Data specialization: Specialized operations on domain-specific data types can do in one cycle what may take tens of cycles on a conventional computer. Specialized logic to perform an inner-loop function gains in both performance and efficiency.

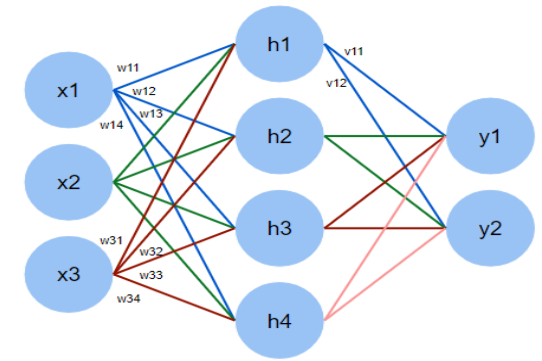
Parallelism: High degrees of parallelism, often exploited at several levels, provide gains in performance. To be effective, the parallel units must exploit locality and make very few global memory references or their performance will be memory bound.

Local and optimized memory: By storing key data structures in many small, local memories, very high memory bandwidth can be achieved with low cost and energy. Access patterns to global memory are optimized to achieve the greatest possible memory bandwidth. Key data structures may be compressed to multiply bandwidth. Memory accesses are load-balanced across memory channels and carefully scheduled to maximize memory utilization.

Reduced overhead: Specializing hardware eliminates or reduces the overhead of program interpretation.

Hardware Acceleration of ML

Deep Neural Networks



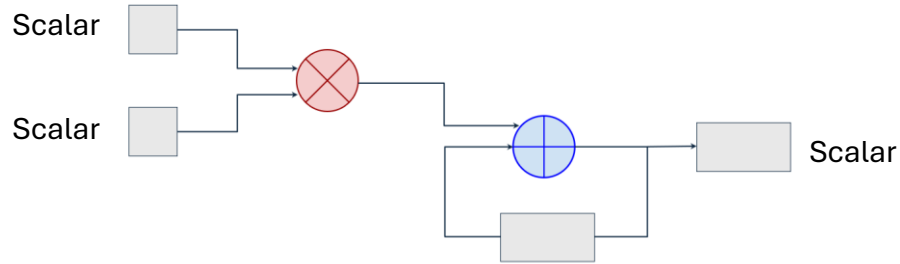
The most prevalent computation in DNNs is....

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix}_{2 \times 3} \times \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}_{3 \times 3} = \begin{bmatrix} y_{11} & y_{12} & y_{13} \\ y_{21} & y_{22} & y_{23} \end{bmatrix}_{2 \times 3}$$

Weight Matrix
Input Matrix
Output Matrix

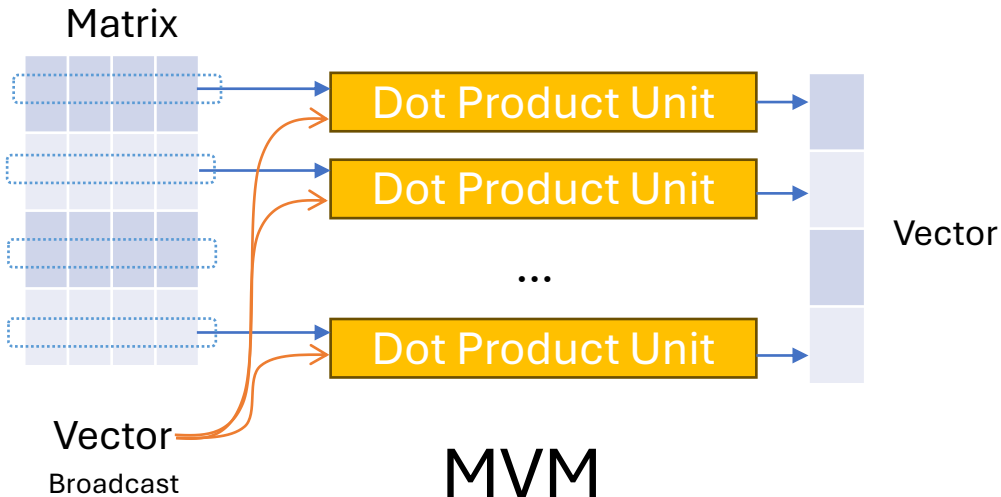
.... matrix-matrix multiplication!

Processing Element



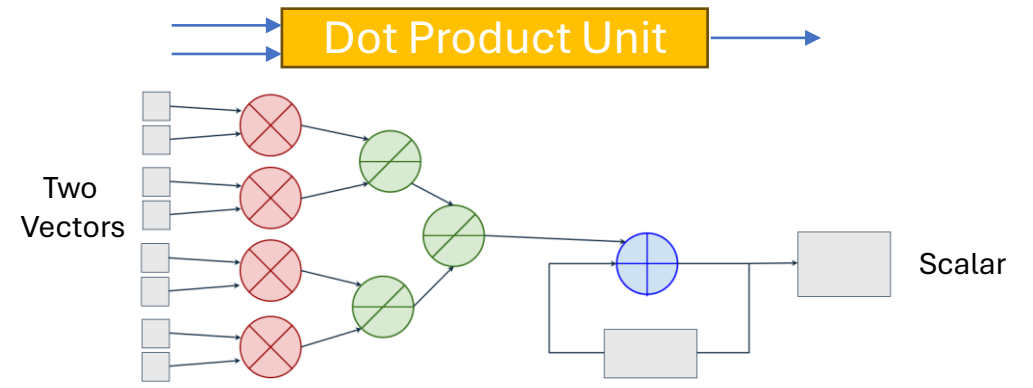
MAC

Many independent MACs



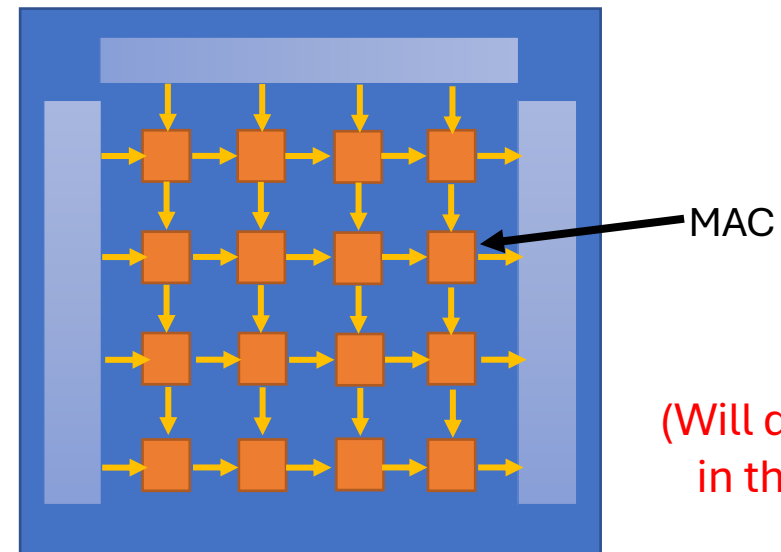
MVM

Many independent MVMs



Dot Product Unit

Many independent Dot Product Units



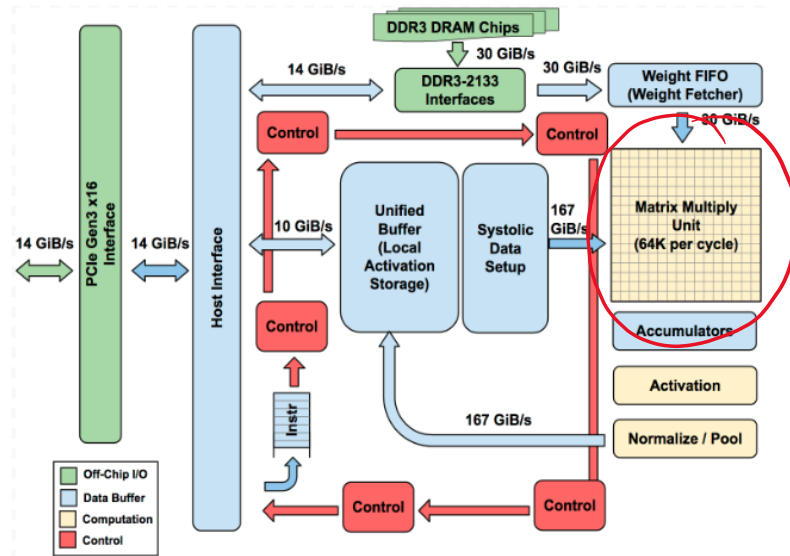
MMM

Many independent MMMs

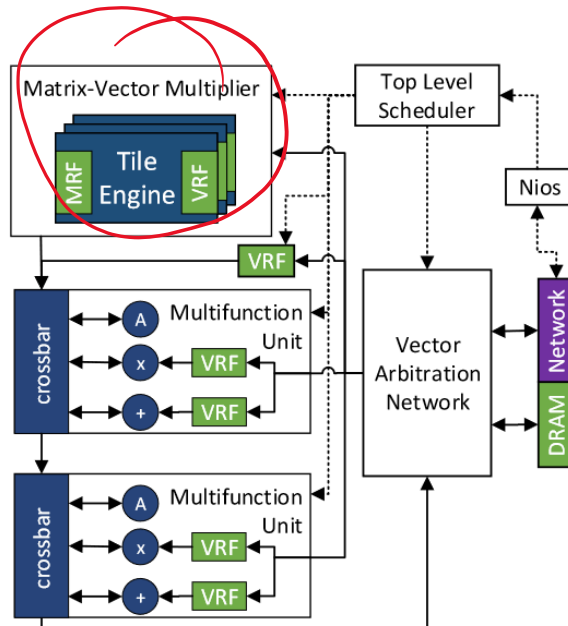
(Will discuss later
in this lecture)

Processing Element

Google TPU
(ASIC)



Microsoft
Brainwave
(FPGA)



NVIDIA GPU



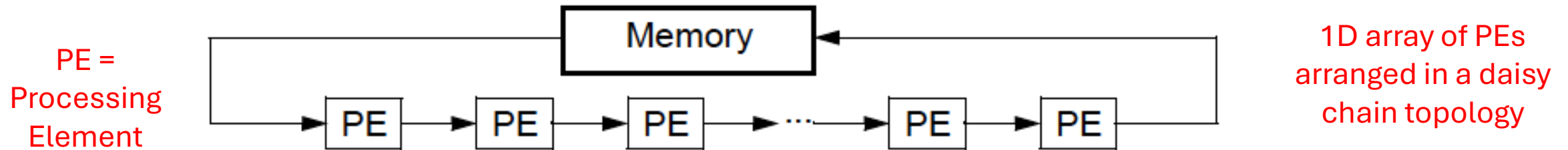
$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

Added a new instruction called HMMA

Systolic Arrays

Definition: A *systolic array* is a network of processors that rhythmically compute and pass data through the system.



Data item is not only used when it is input but also *reused* as it moves through the pipelines in the array.

Thoroughly investigated in eighties [**IEEE Computer**, Jan. 1982, **July 1987**].

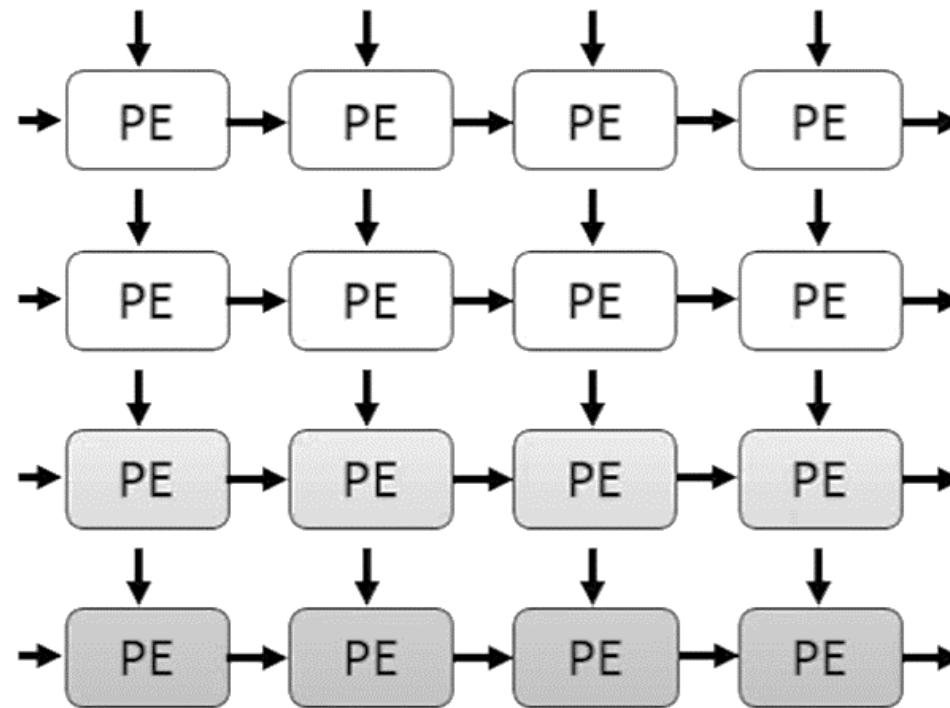
“Systolic array” - analogy with the human circulatory system:

- ◆ heart == global memory,
- ◆ network of veins == array of processors and links.

Systolic Arrays

2D array of PEs arranged in a mesh topology

Data enters/leaves at mesh boundaries and is communicated with neighboring PEs while being processed at each PE.



Very efficient use of memory

Reduced wire usage. No global wires.

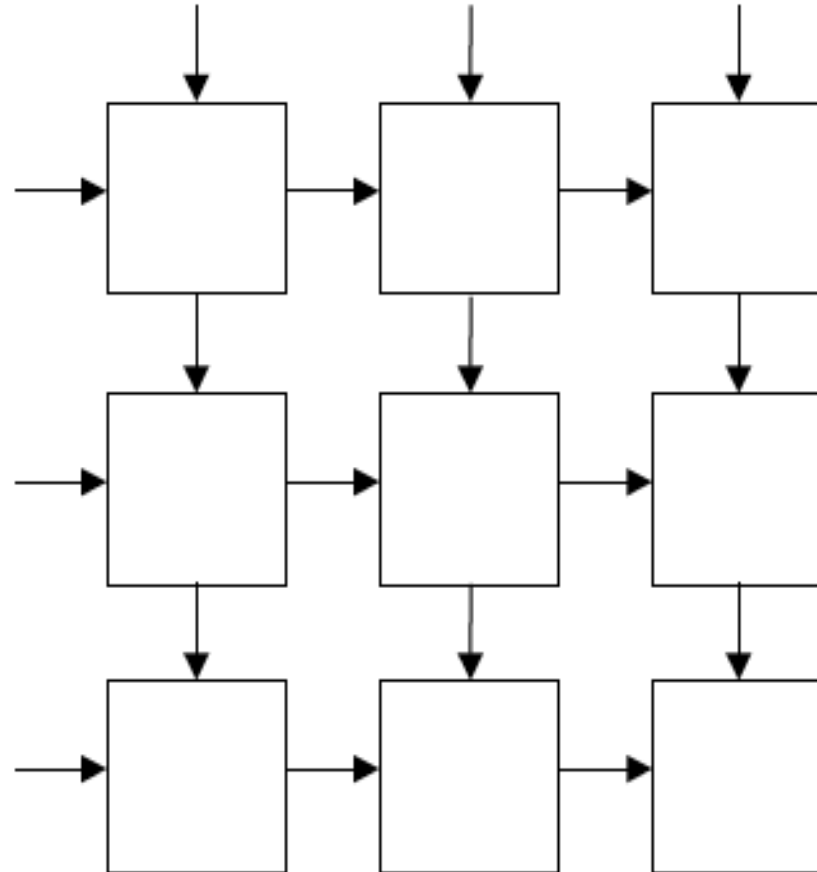
Inner PEs only have near-neighbor connections.

Very symmetric dataflow

3 X 3 Matrix Multiply Example

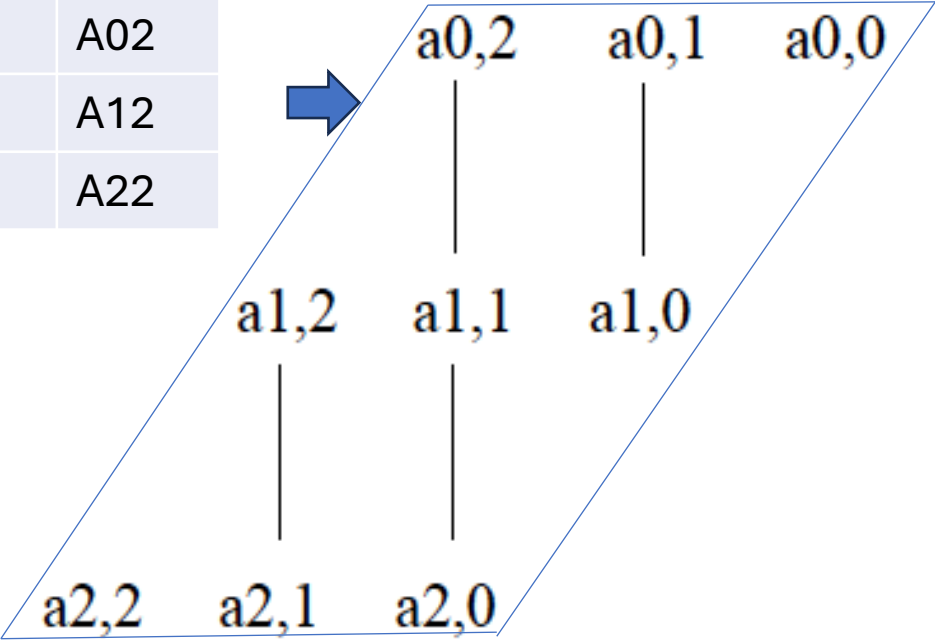
B00	B01	B02
B10	B11	B12
B20	B21	B22

A00	A01	A02
A10	A11	A12
A20	A21	A22

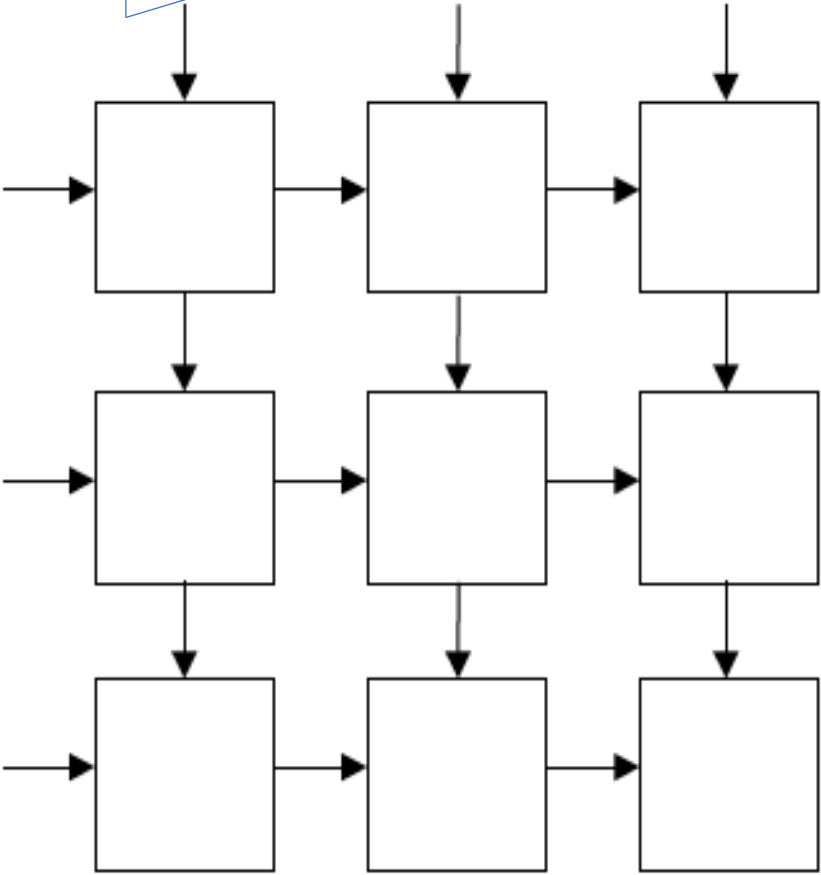
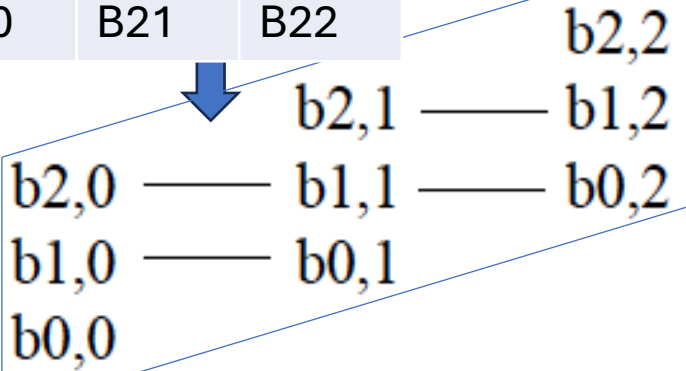


3 X 3 Matrix Multiply Example

A00	A01	A02
A10	A11	A12
A20	A21	A22

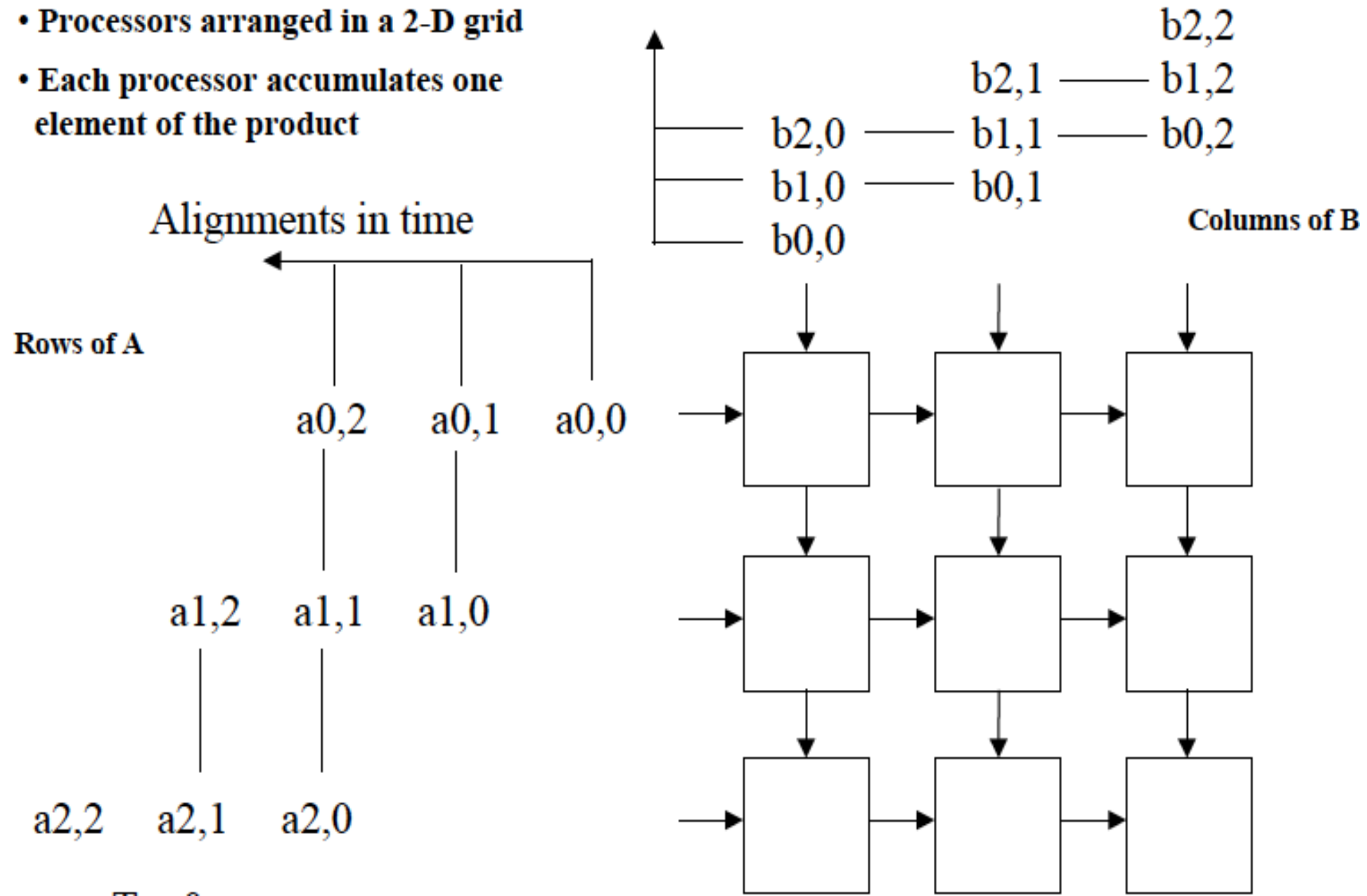


B00	B01	B02
B10	B11	B12
B20	B21	B22



3 X 3 Matrix Multiply Example Time T = 0

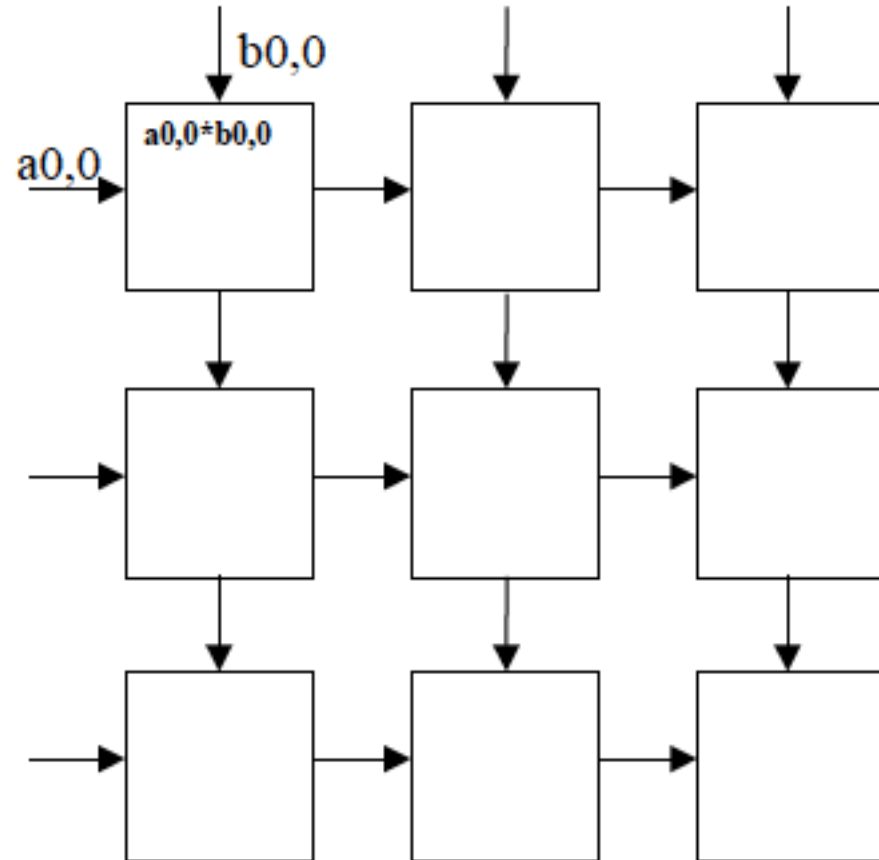
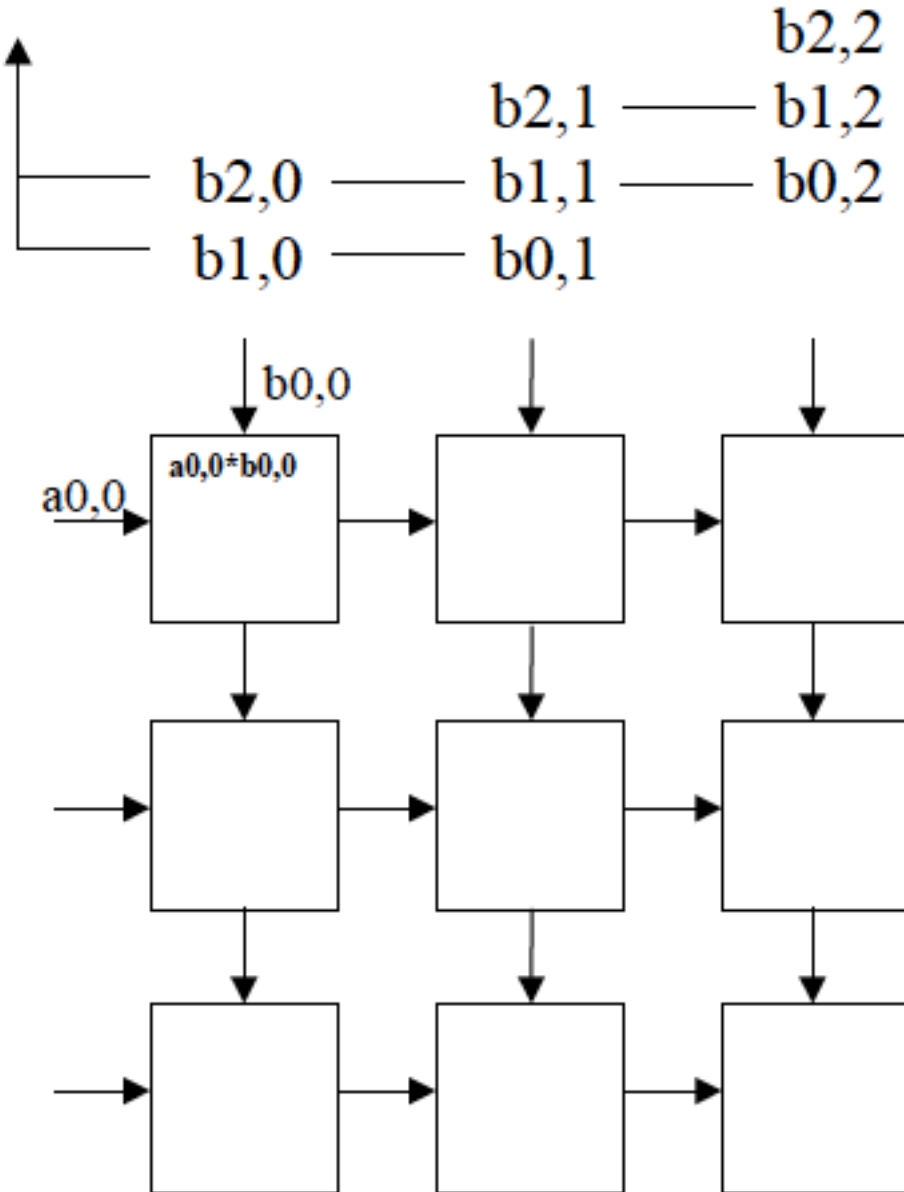
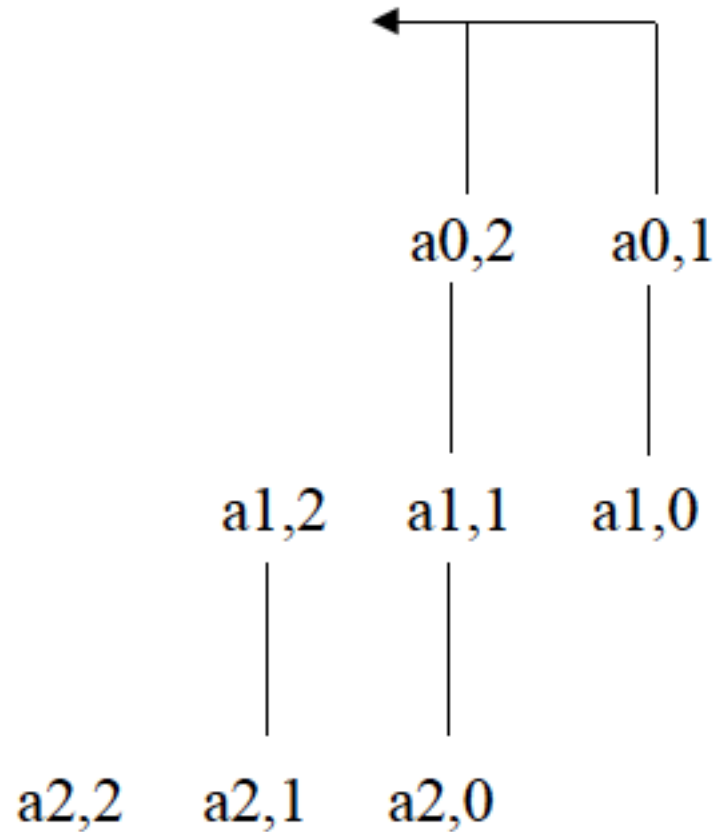
- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



3 X 3 Matrix Multiply Example Time T = 1

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

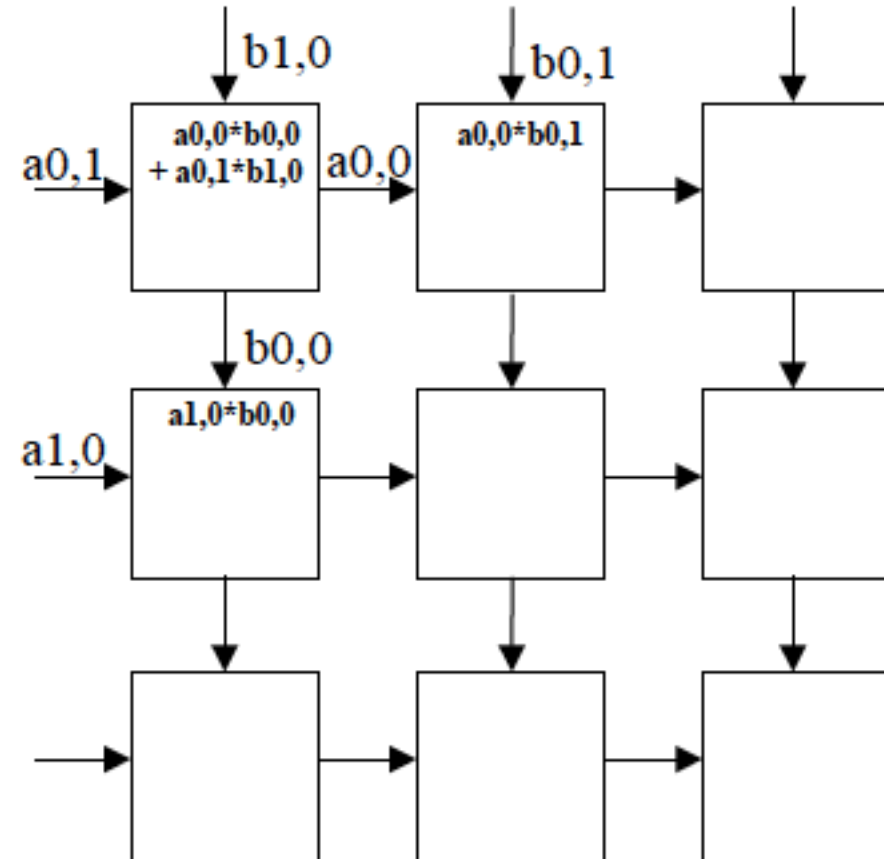
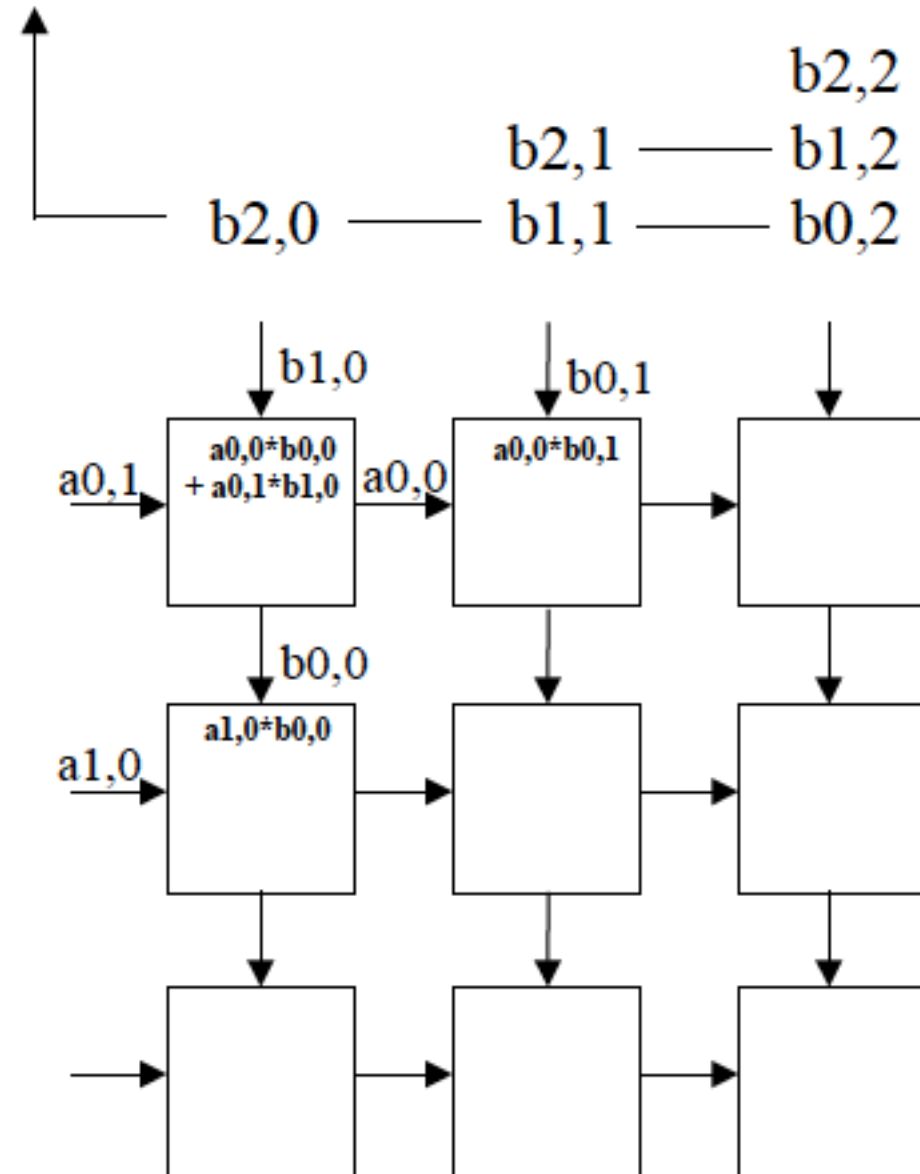
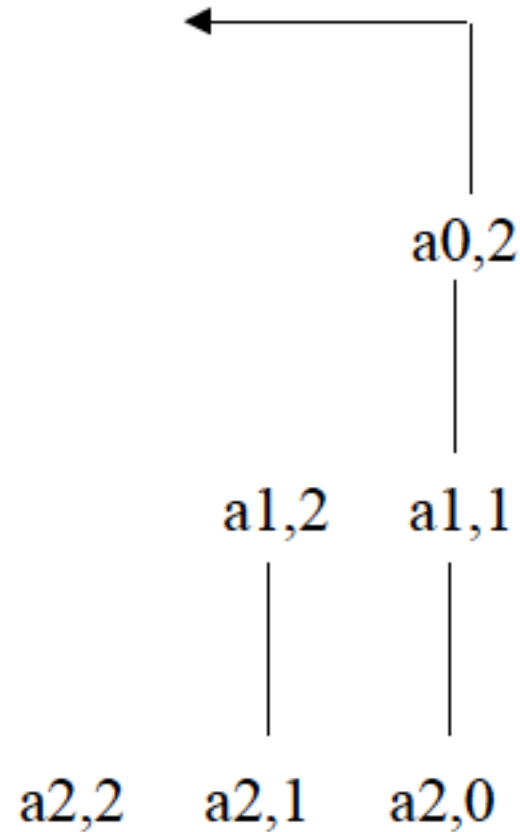
Alignments in time



3 X 3 Matrix Multiply Example Time T = 2

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

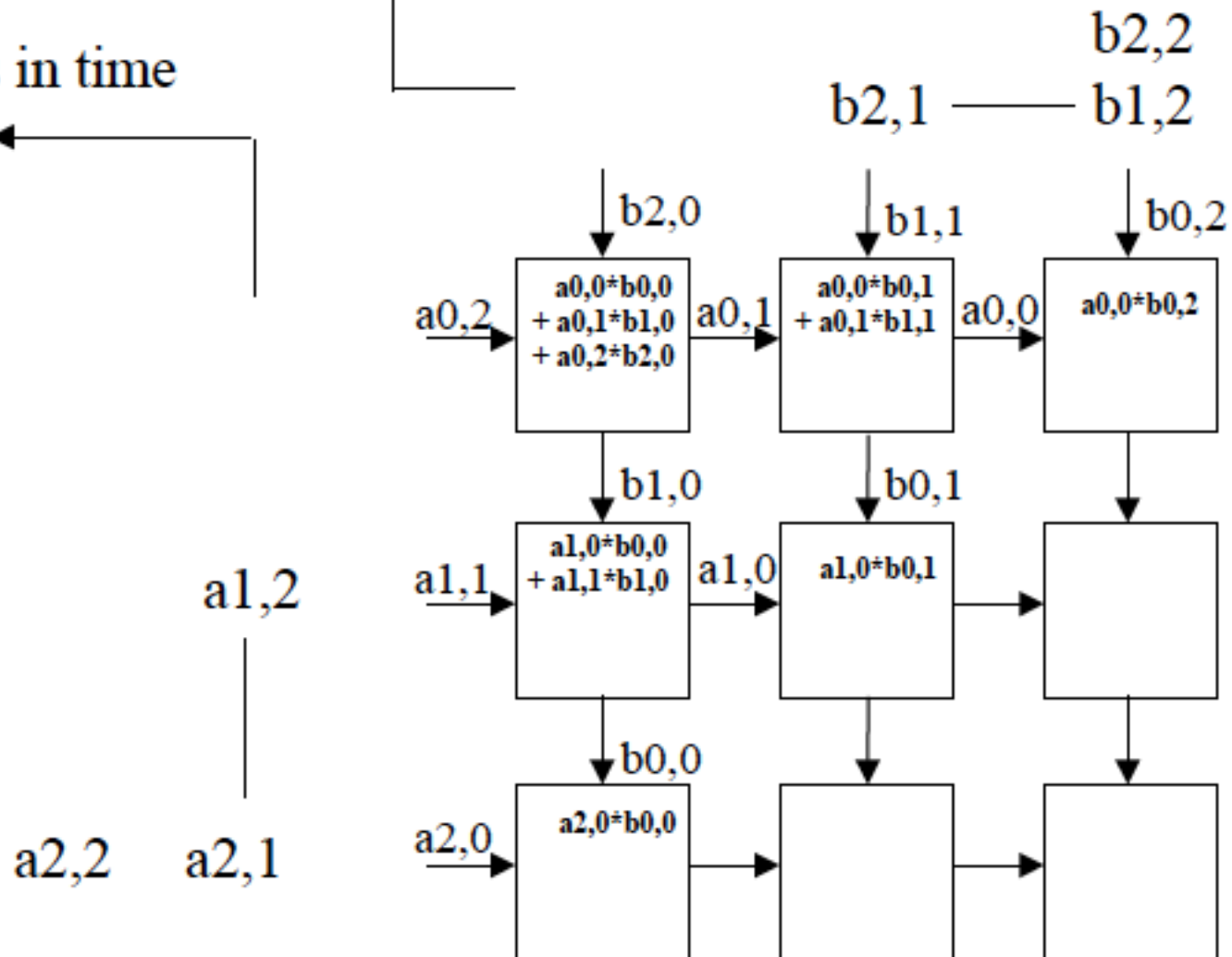
Alignments in time



3 X 3 Matrix Multiply Example Time T = 3

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

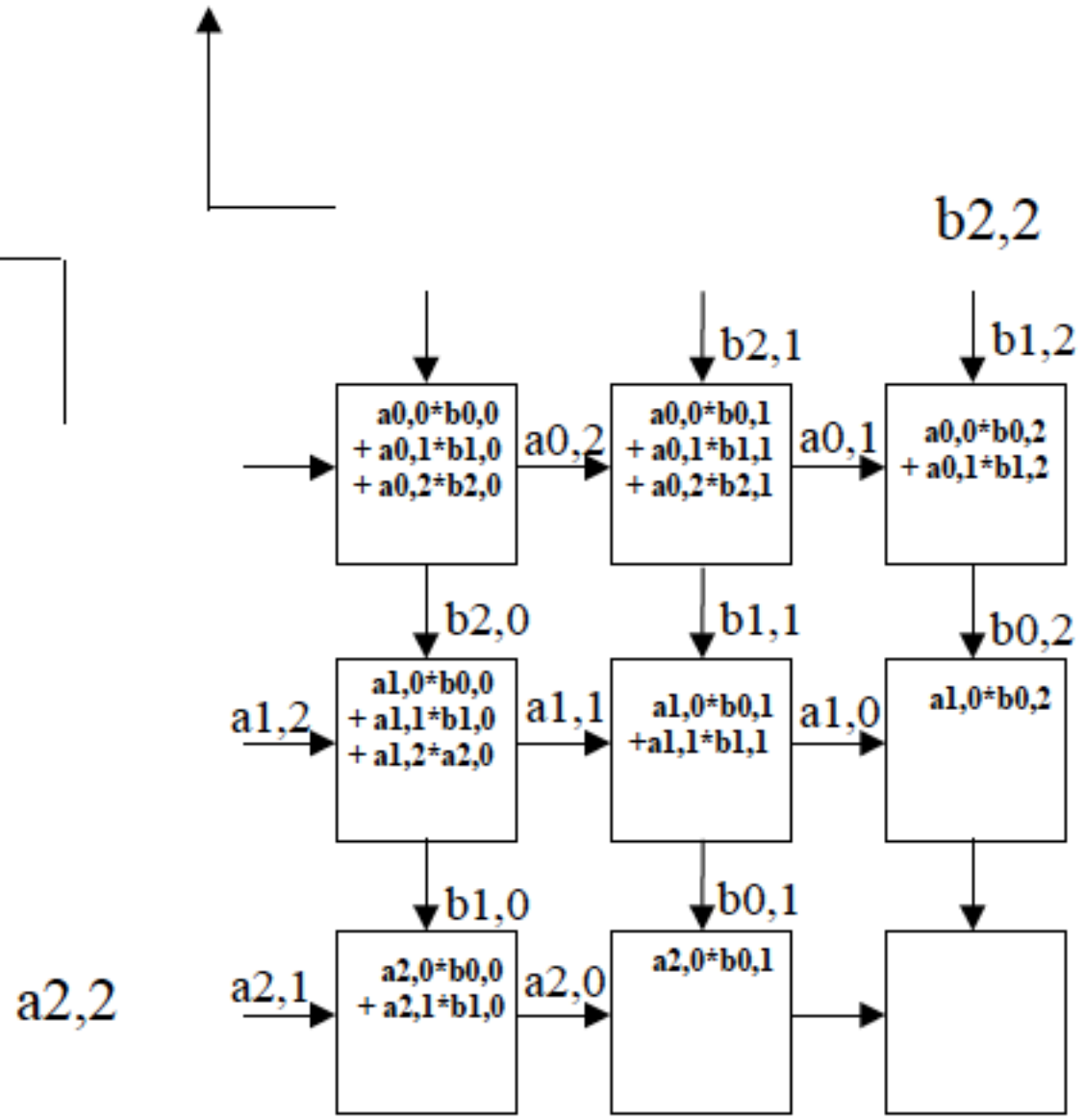
Alignments in time



3 X 3 Matrix Multiply Example Time T = 4

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

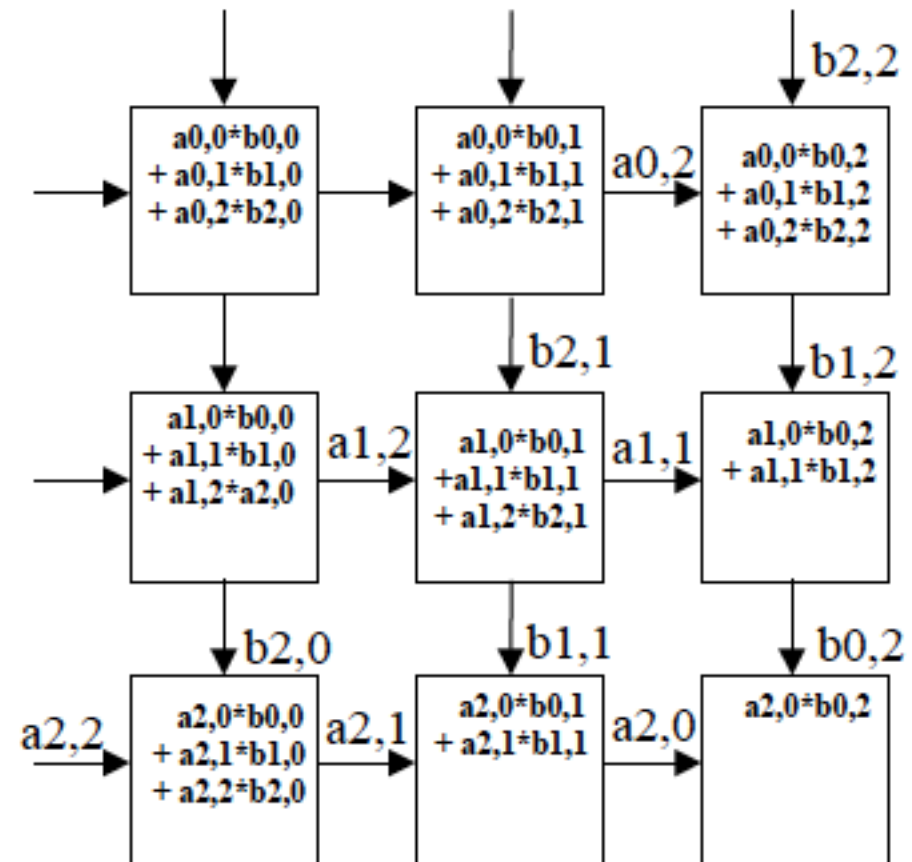
Alignments in time



3 X 3 Matrix Multiply Example Time T = 5

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

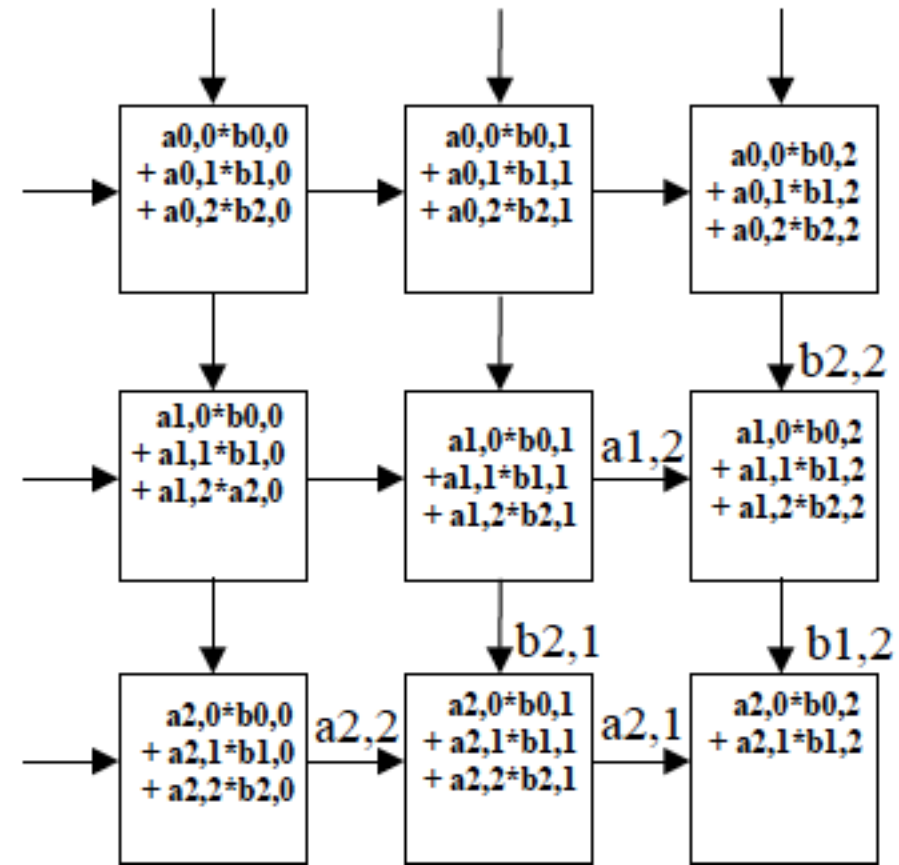
Alignments in time



3 X 3 Matrix Multiply Example Time T = 6

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

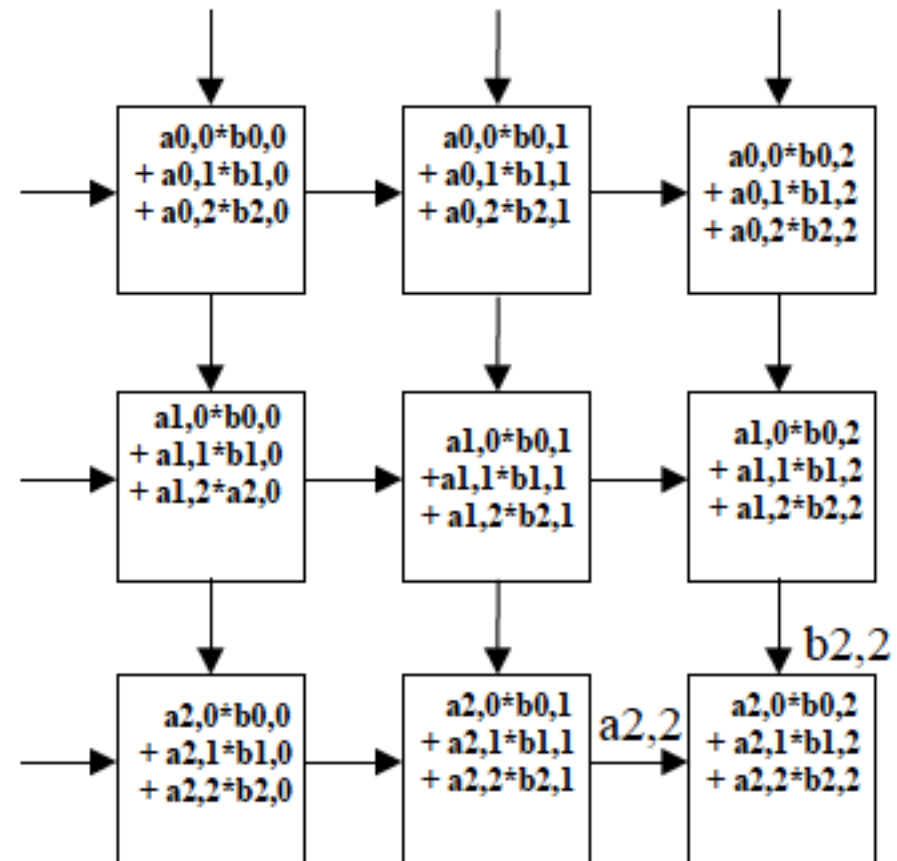
Alignments in time



3 X 3 Matrix Multiply Example Time T = 7

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

Alignments in time



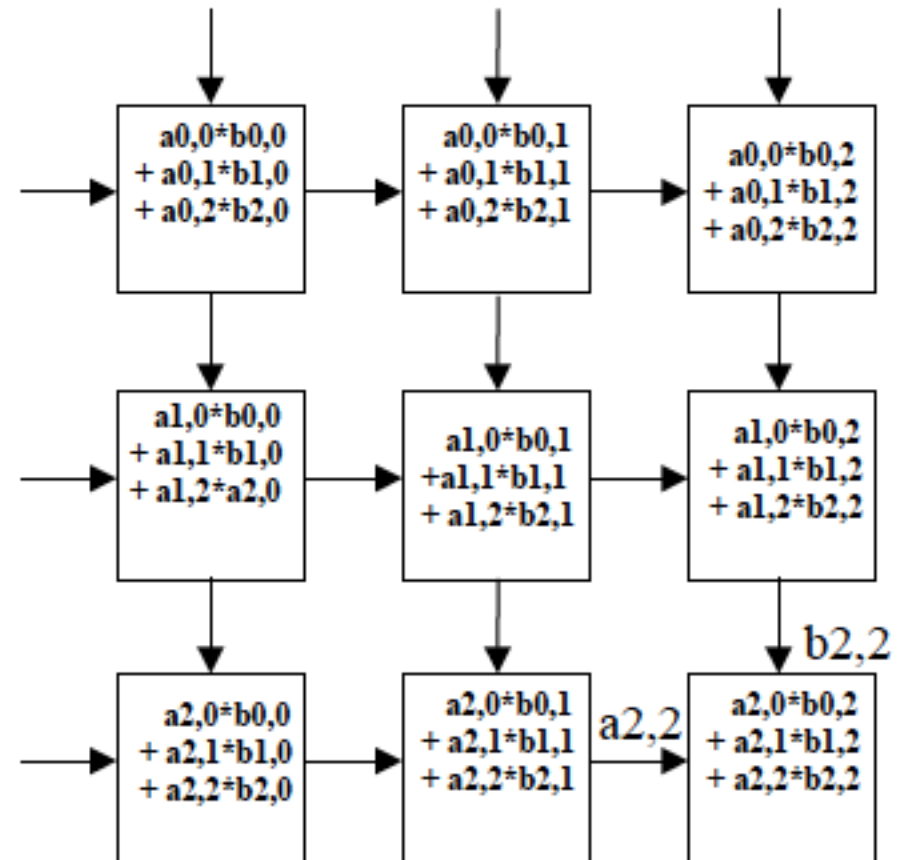
3 X 3 Matrix Multiply Example Time T = 7

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product

Alignments in time

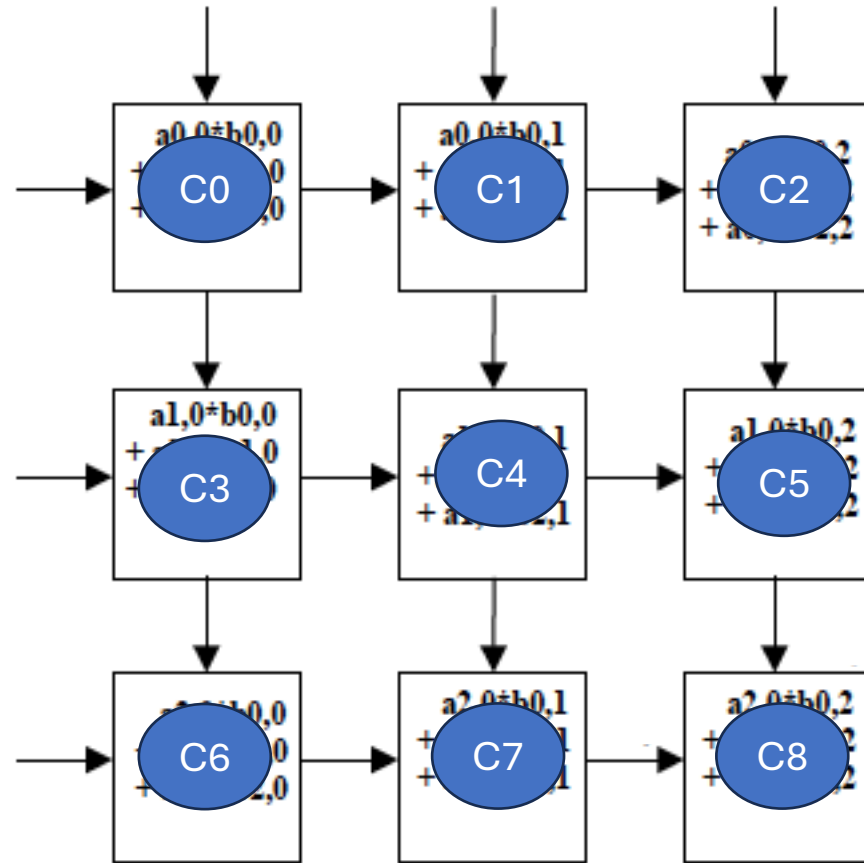
A00	A01	A02	B00	B01	B02
A10	A11	A12	B10	B11	B12
A20	A21	A22	B20	B21	B22

Observe what we have in each PE now



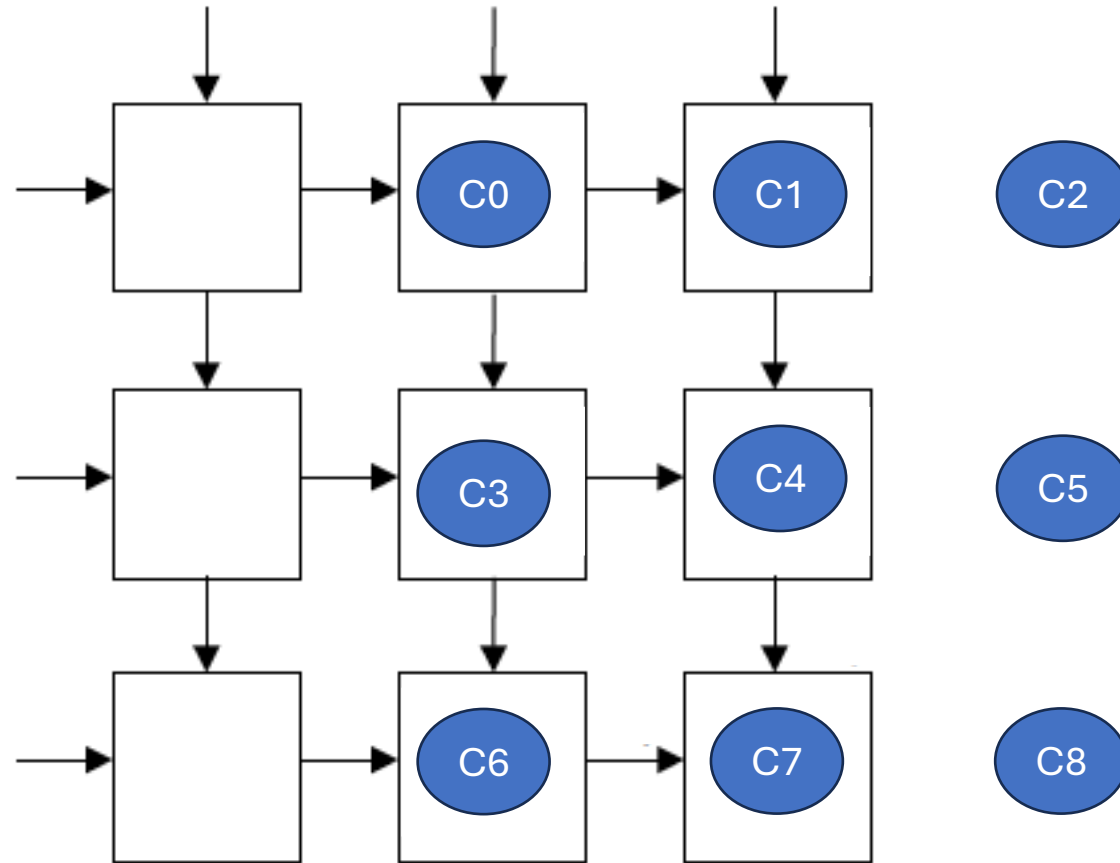
3 X 3 Matrix Multiply Example Time T = 7

Now we have the results in each PE

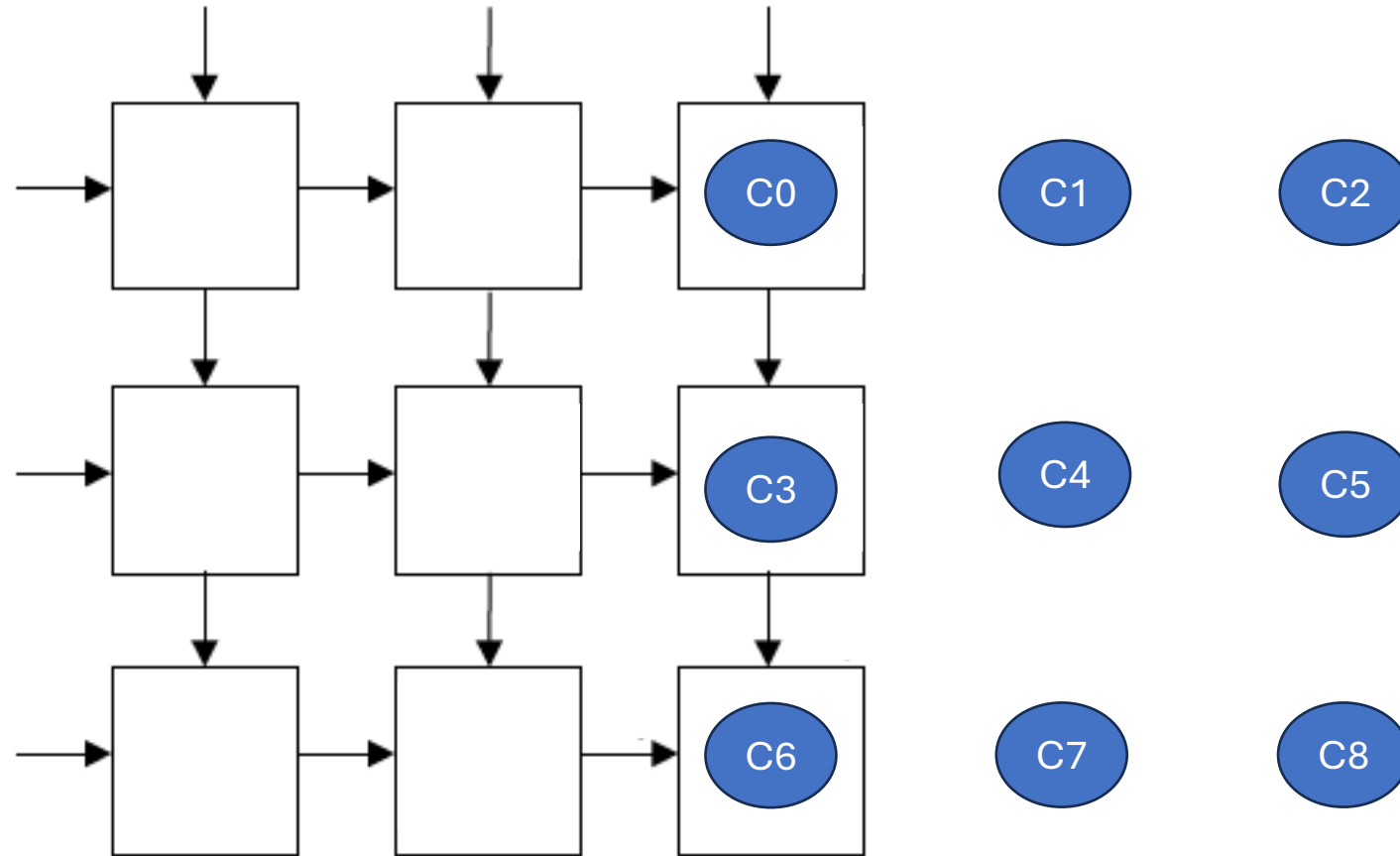


3 more cycles to shift them out

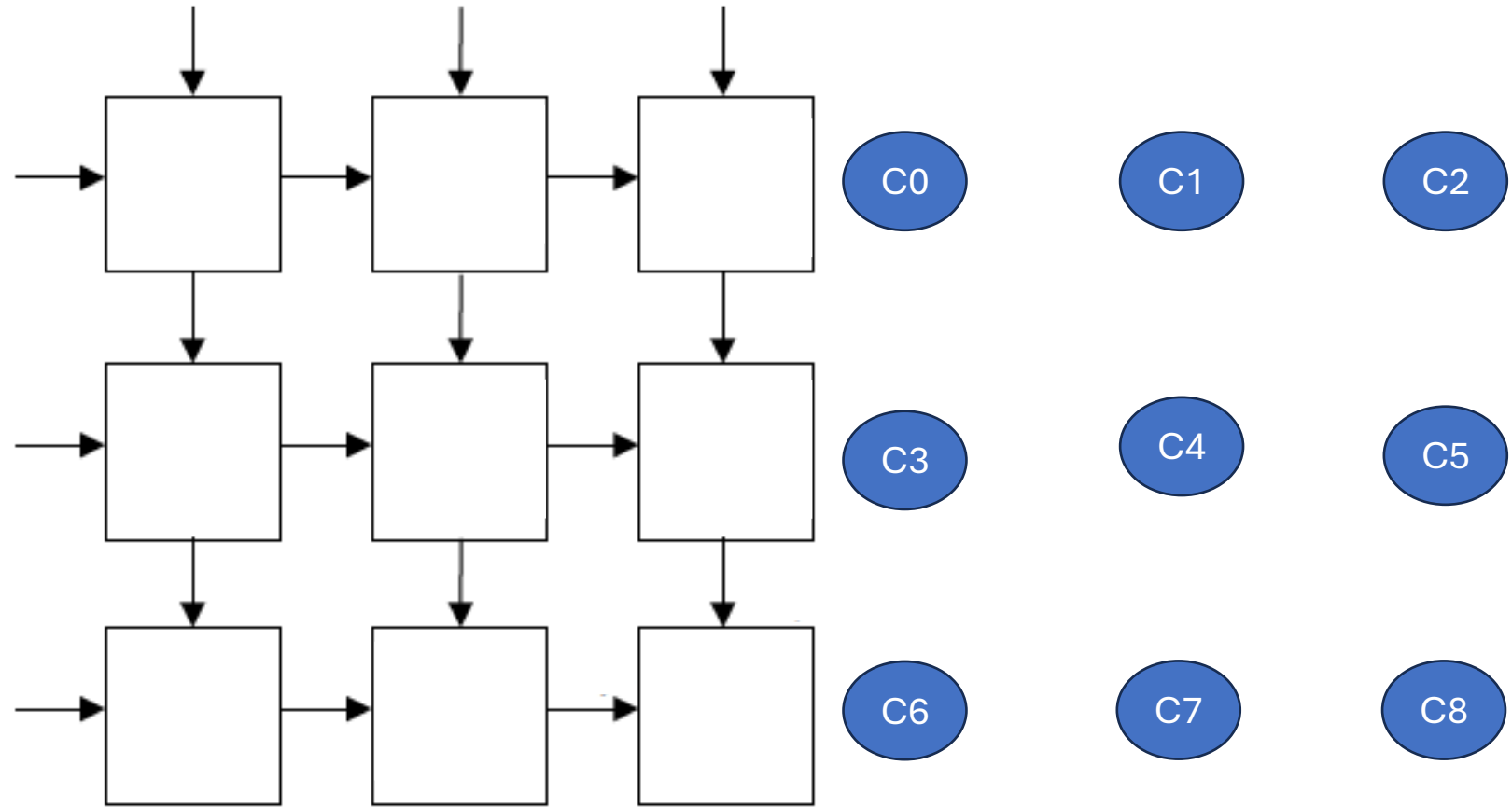
3 X 3 Matrix Multiply Example Time T = 8



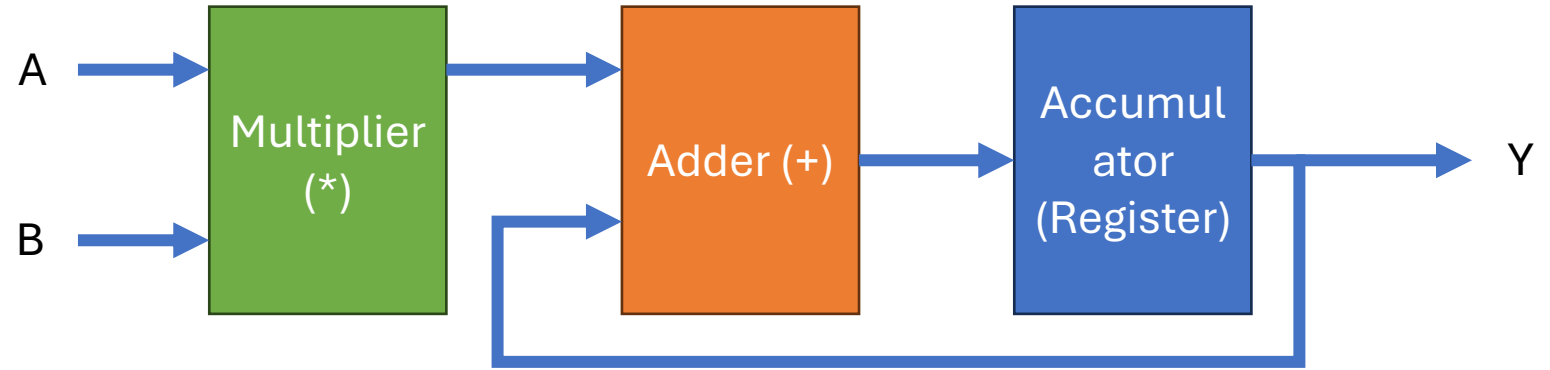
3 X 3 Matrix Multiply Example Time T = 9



3 X 3 Matrix Multiply Example Time T = 10

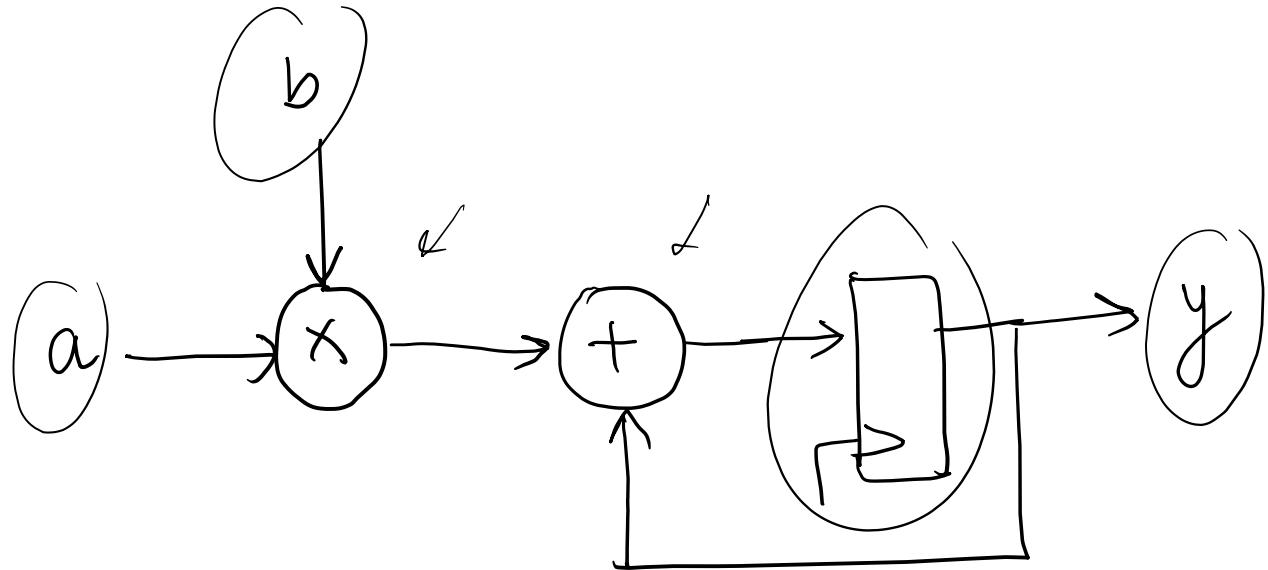


What operation is each PE doing?



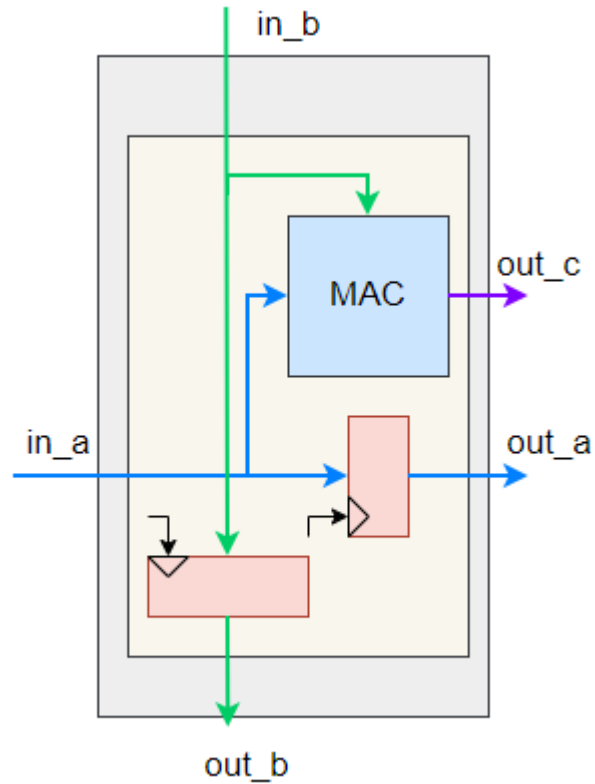
The MAC operation

$$Y = A*B + Y_{\text{prev}}$$



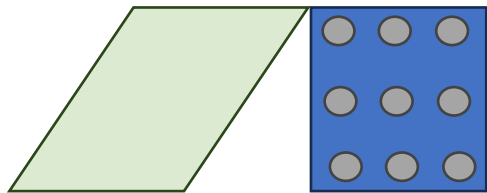
What operation is each PE doing?

And, also, shifting the received data...

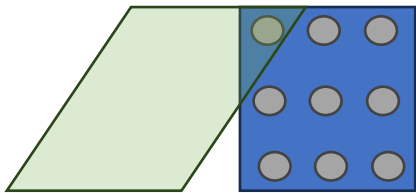


Properties of Systolic Matrix Multiplier

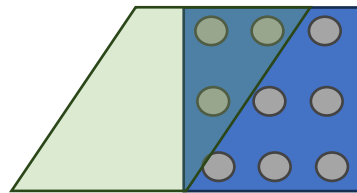
- Number of PEs = N^2
- Cycles taken = $4N-2$ (+Number of cycles in the MAC)
- Number of memory accesses = $3*N^2$
 - N^2 for input matrix A
 - N^2 for input matrix B
 - N^2 for output matrix C
- Peak memory bandwidth required (reads) = $2N$ elements per clock
- Peak memory bandwidth required (writes) = N elements per clock
- Number of MAC operations = N^3



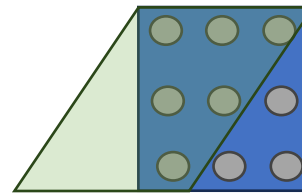
T=0



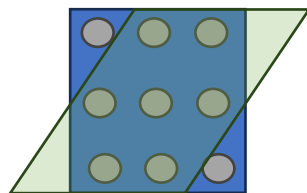
T=1



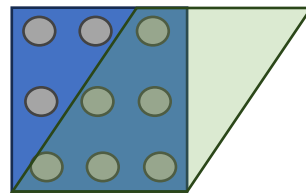
T=2



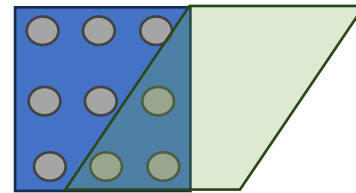
T=3



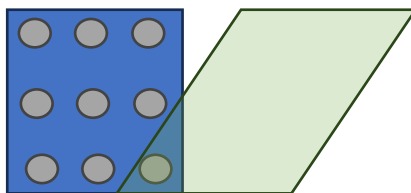
T=4



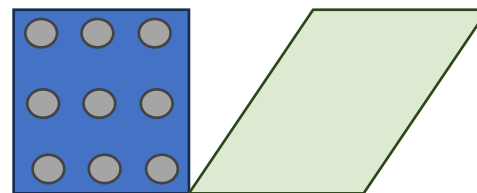
T=5



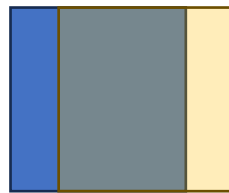
T=6



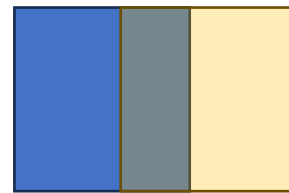
T=7



T=8



T=8



T=9

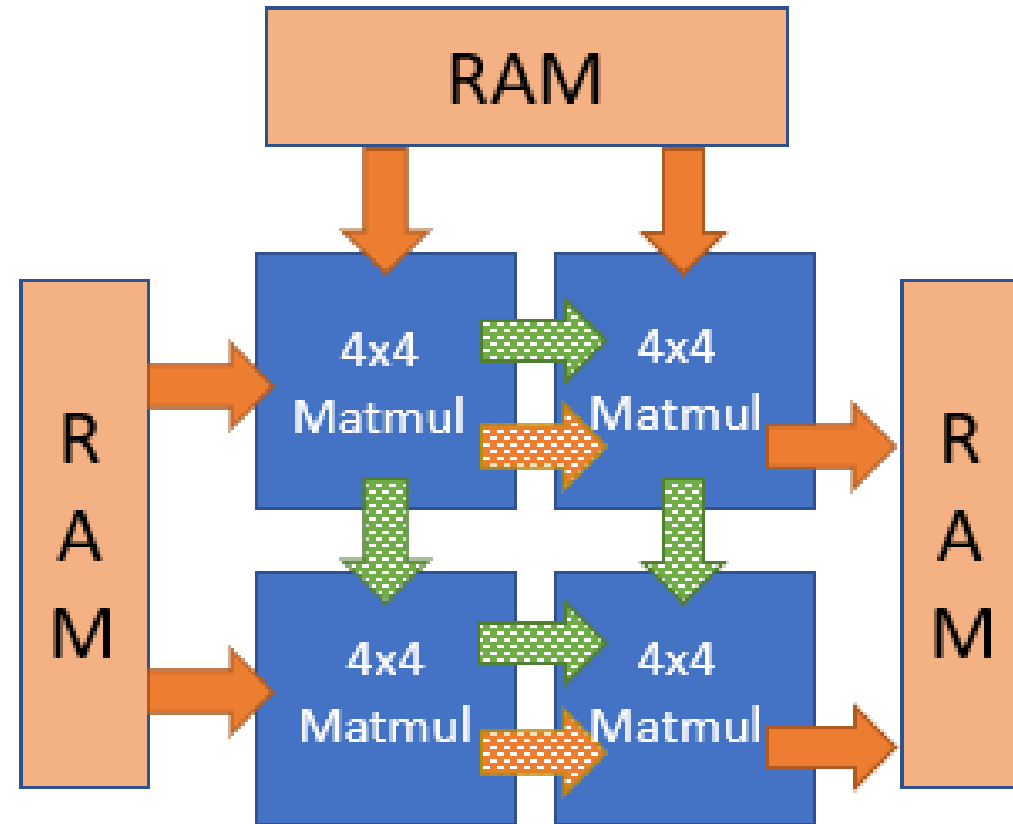


T=10

3x3 Systolic Array for Matrix Multiplication

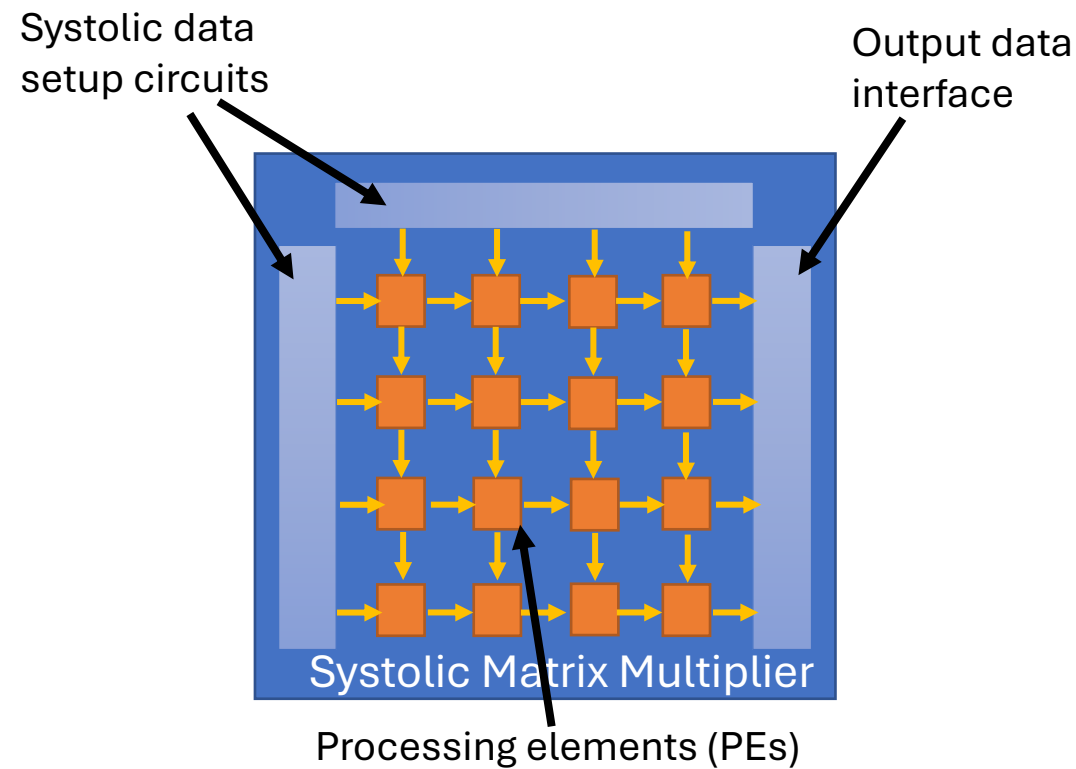


Making Larger Matrix Multipliers

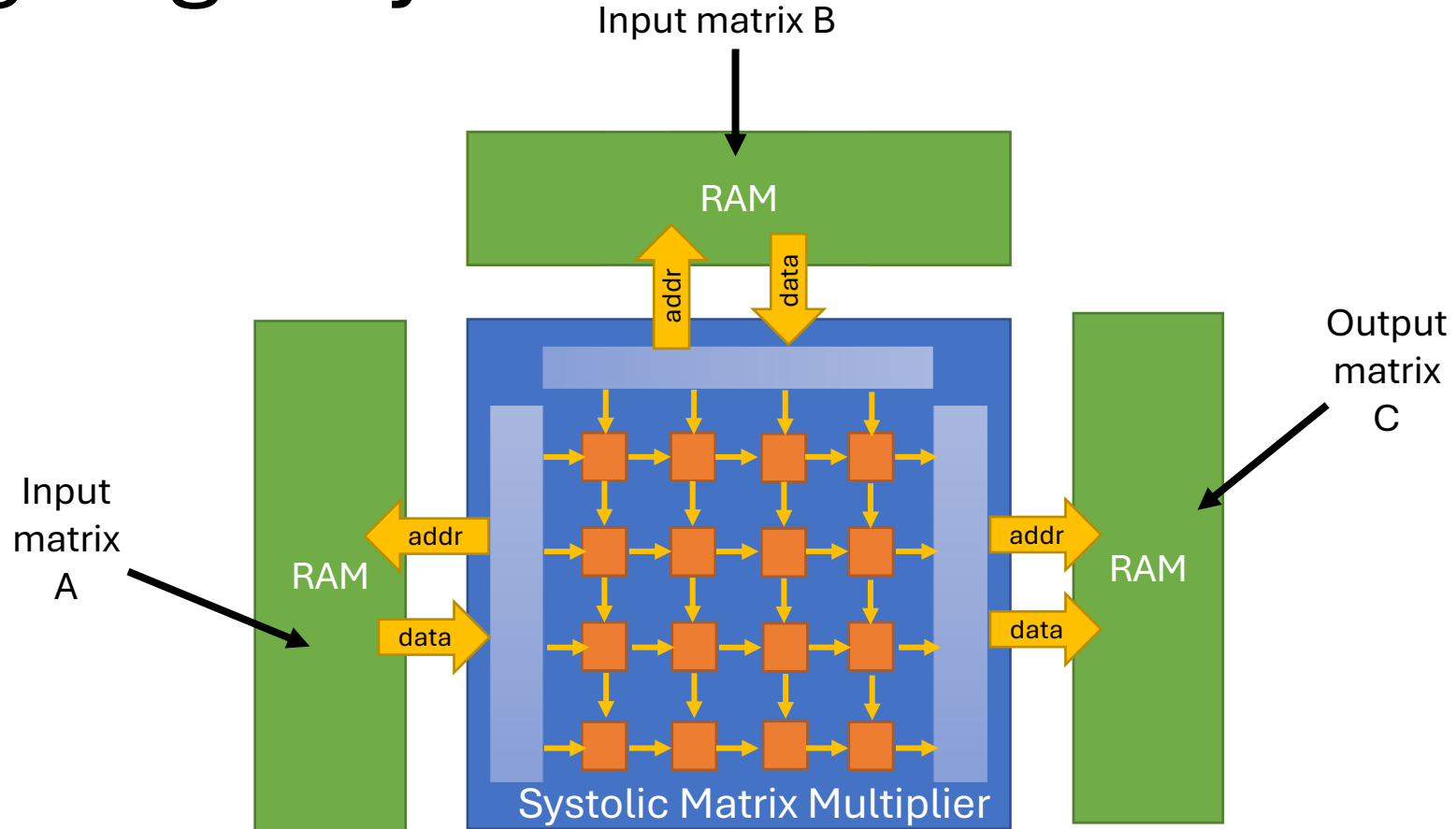


Making a 8x8 matmul using 4 4x4 matmuls

Designing a Systolic MatMul in Verilog



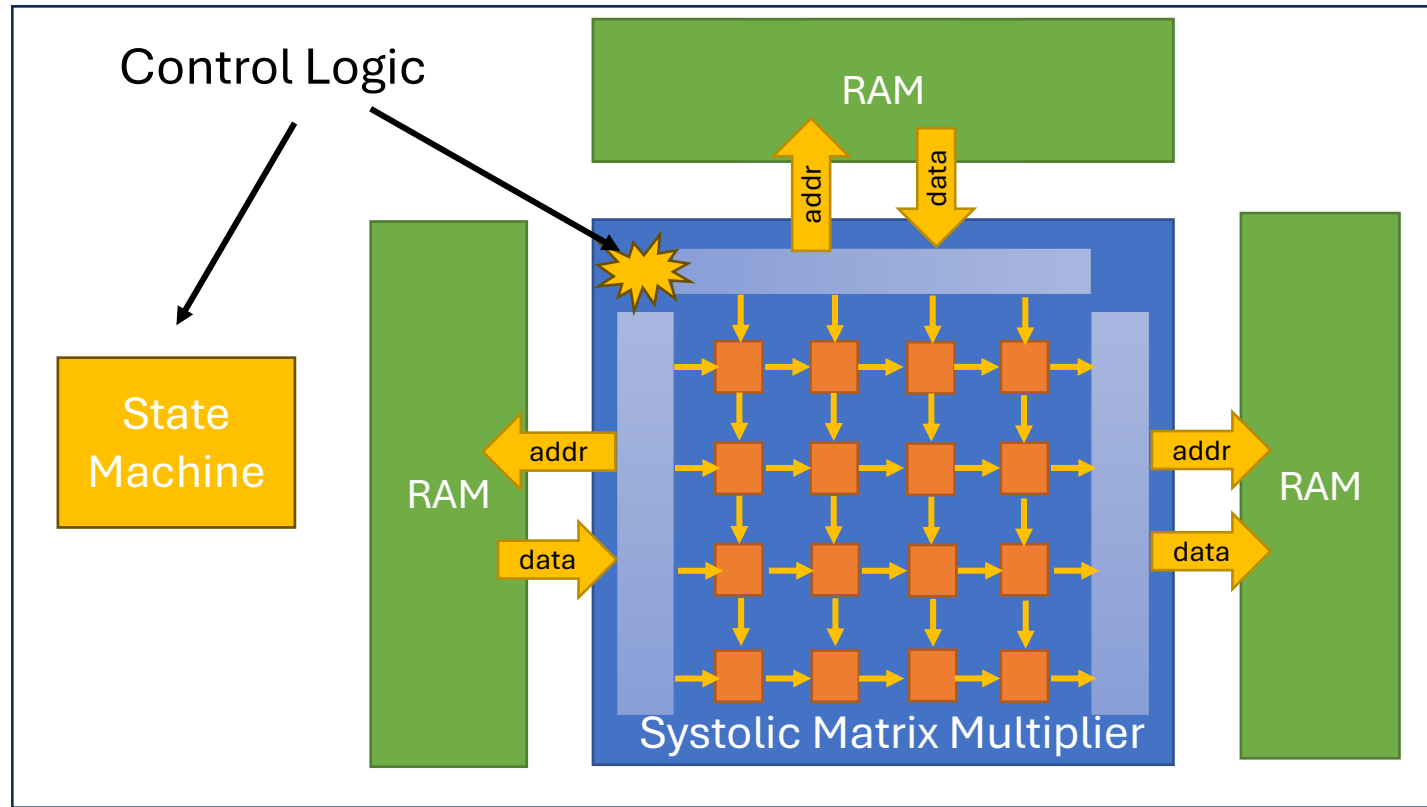
Designing a Systolic MatMul in Verilog



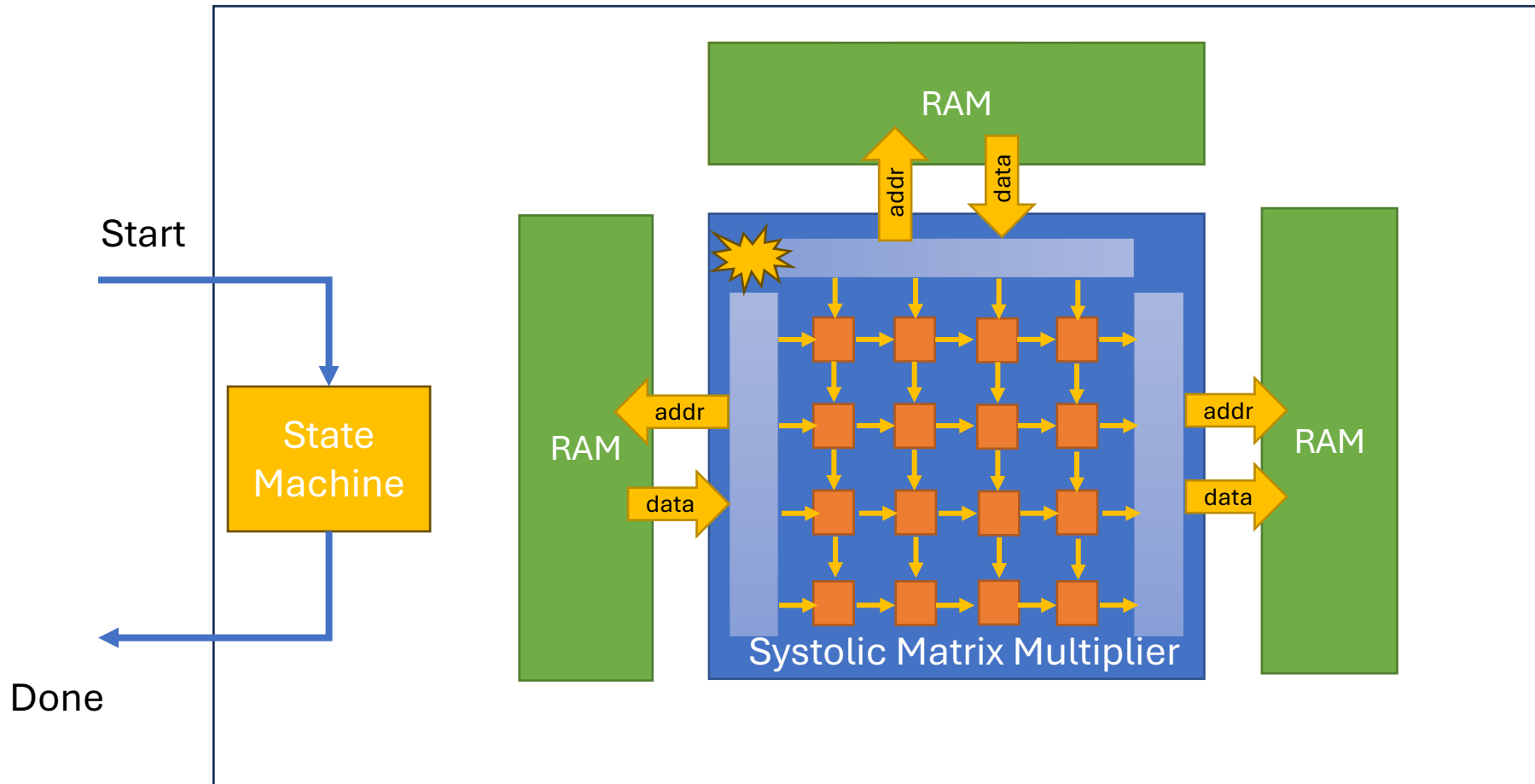
Signals of a RAM:

1. Addr
2. Data In
3. Data Out
4. Write Enable

Designing a Systolic MatMul in Verilog

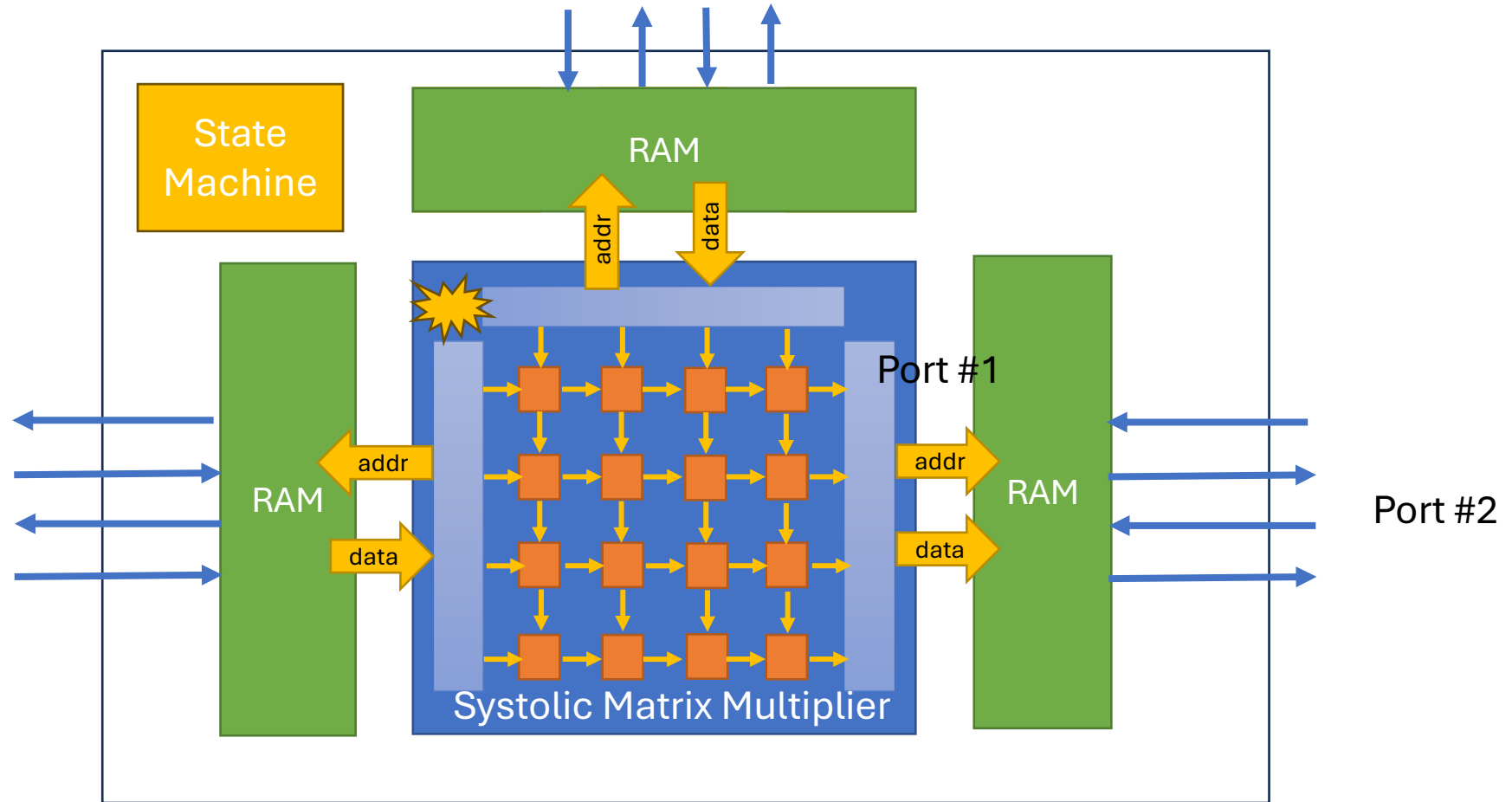


Designing a Systolic MatMul in Verilog



Designing a Systolic MatMul in Verilog

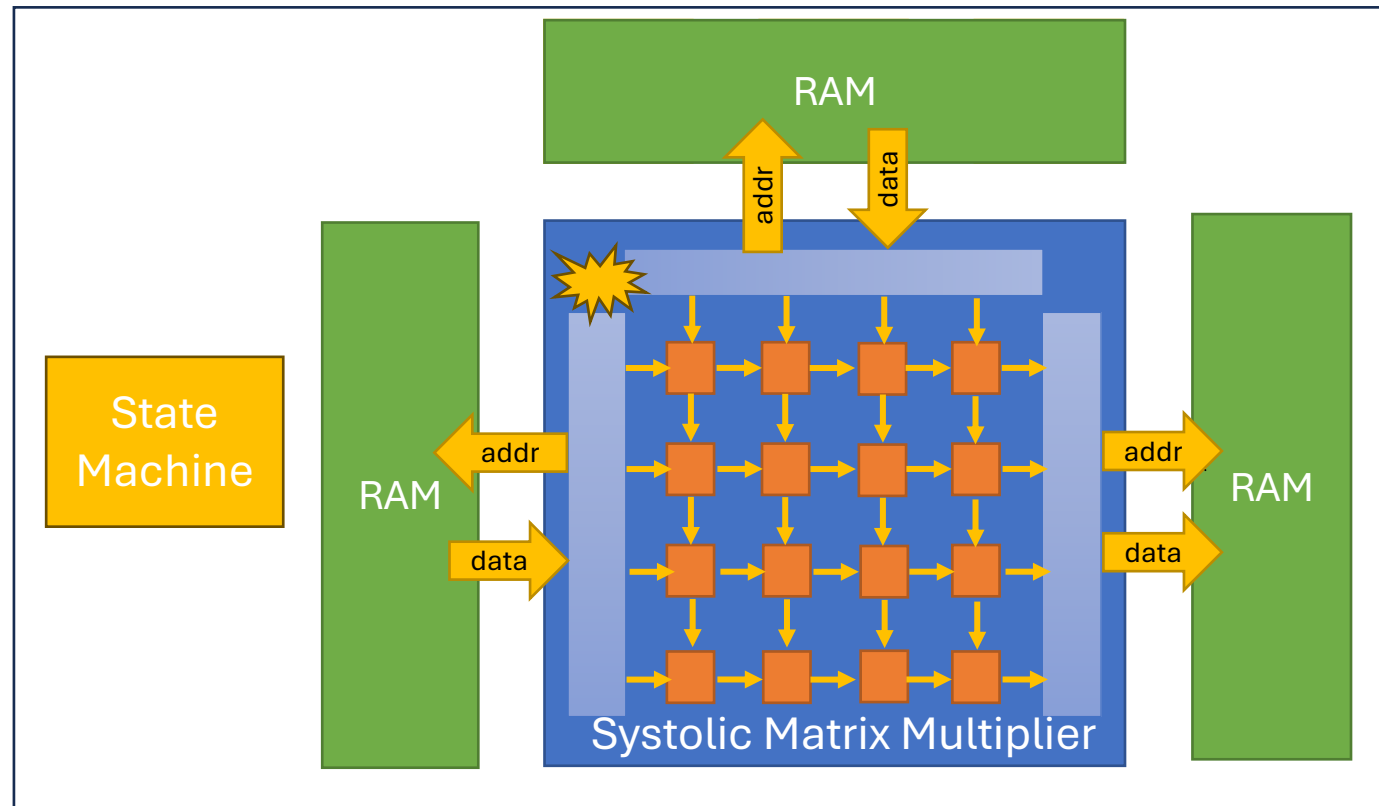
- Dual Ported Memories
- Port 1: Connected to matmul
 - Signals not needed are tied off
- Port 2: Connected to external world
 - Loading/unloading these RAMs
 - `addr_ext`, `wdata_ext`, `rdata_ext`, `we_ext`



Designing a Systolic MatMul in Verilog

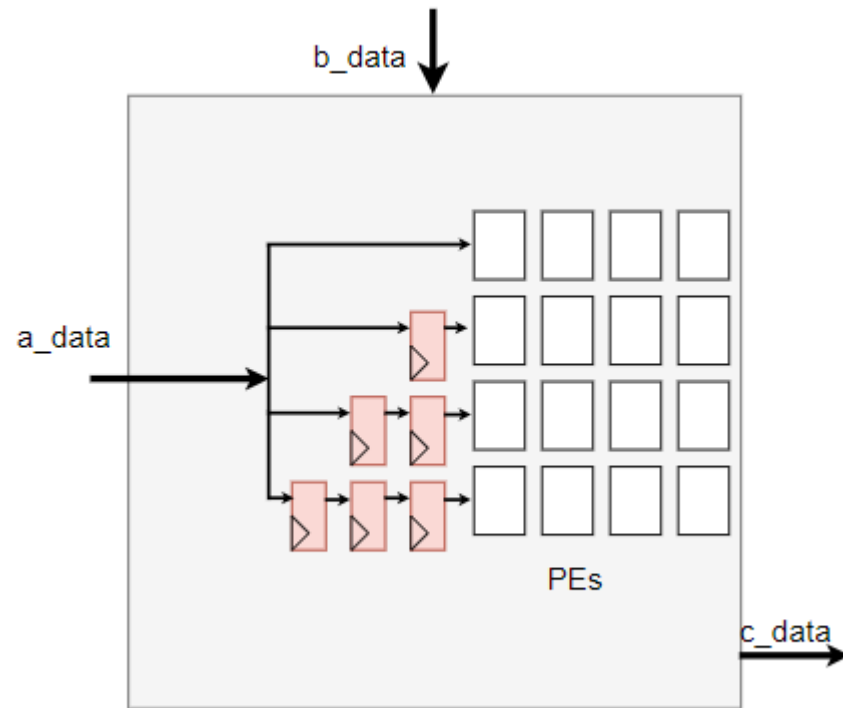
Testbench Code:

1. Load input memories
2. Deassert Reset
3. Trigger Start
4. Wait for Done
5. Compare results
6. Finish

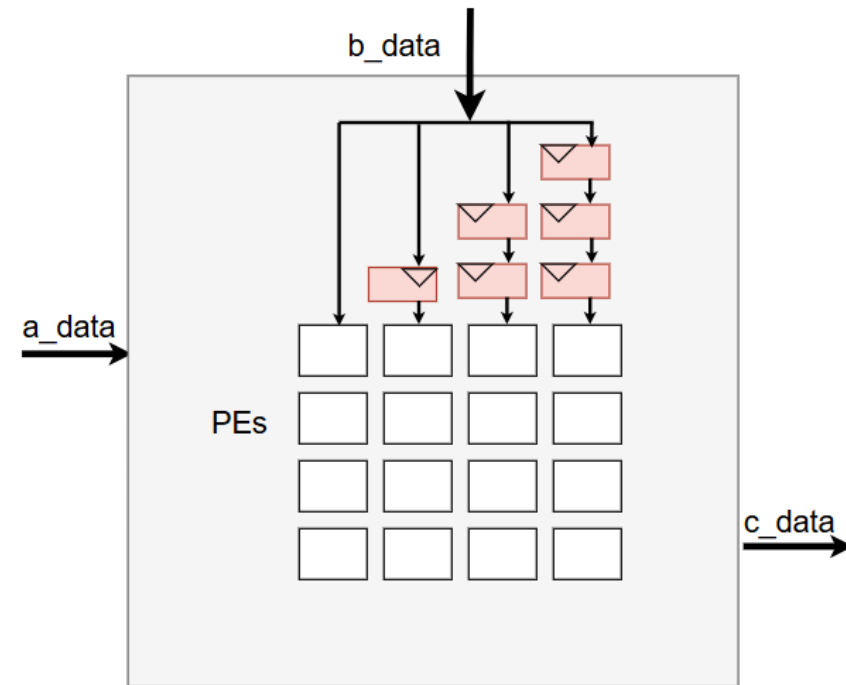


Testbench

Designing a Systolic MatMul in Verilog

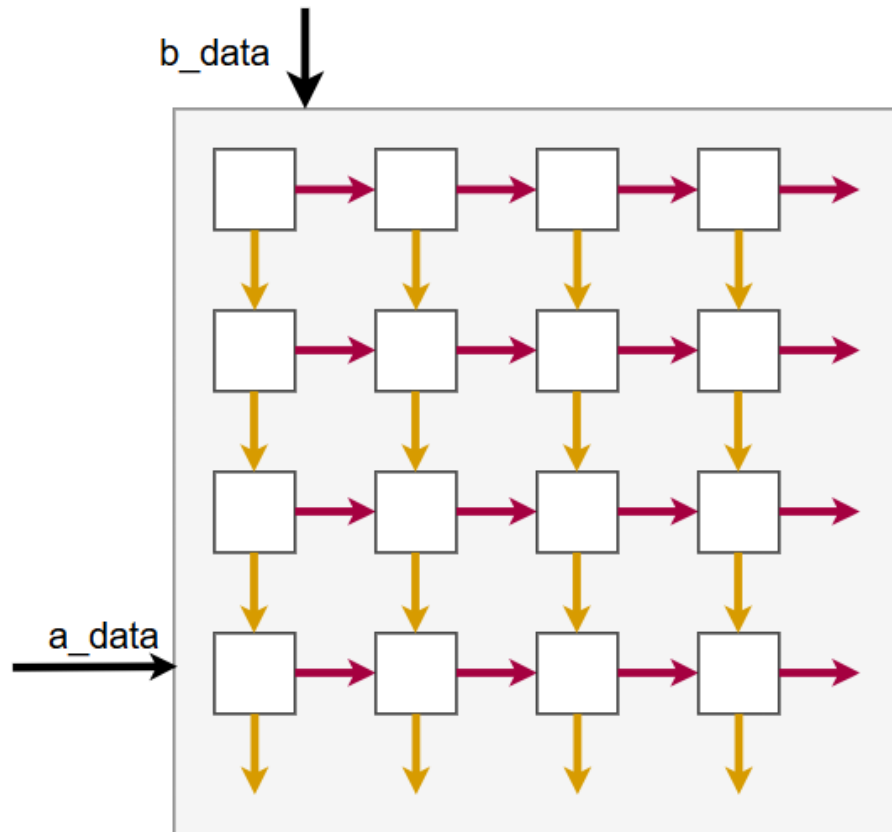


(a) Systolic data setup for Matrix A

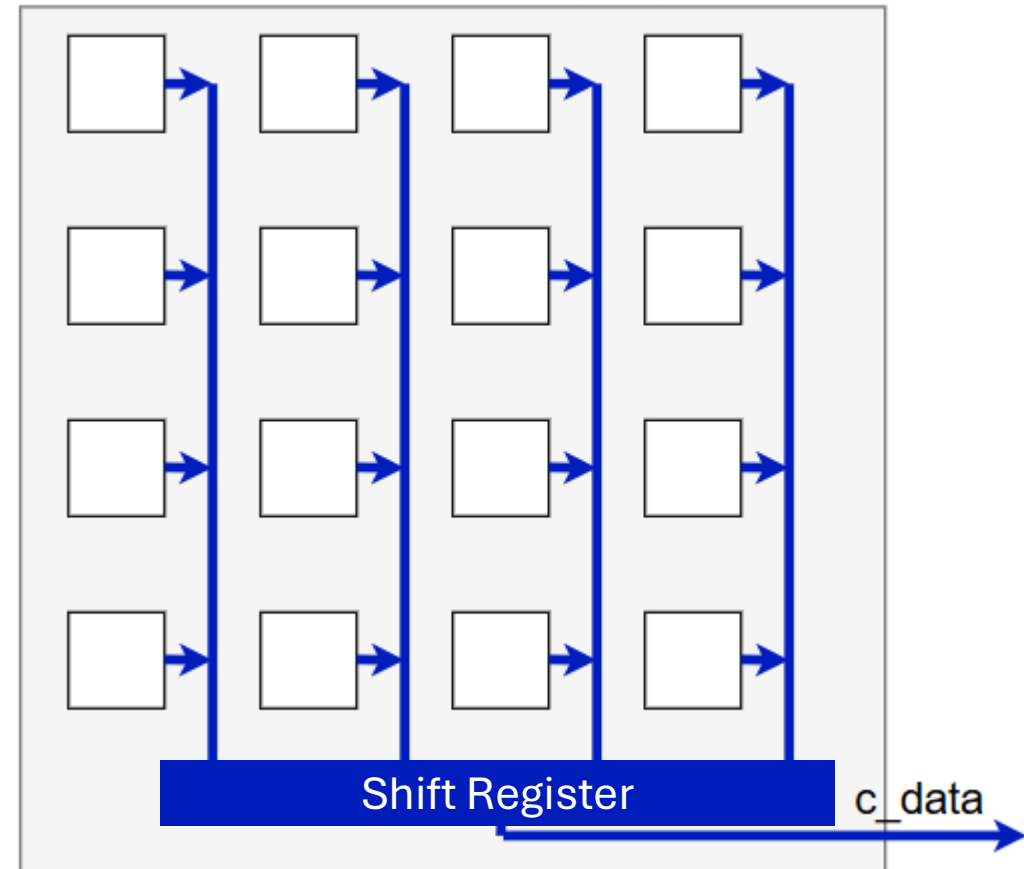


(b) Systolic data setup for Matrix B

Designing a Systolic MatMul in Verilog



Movement of input matrices

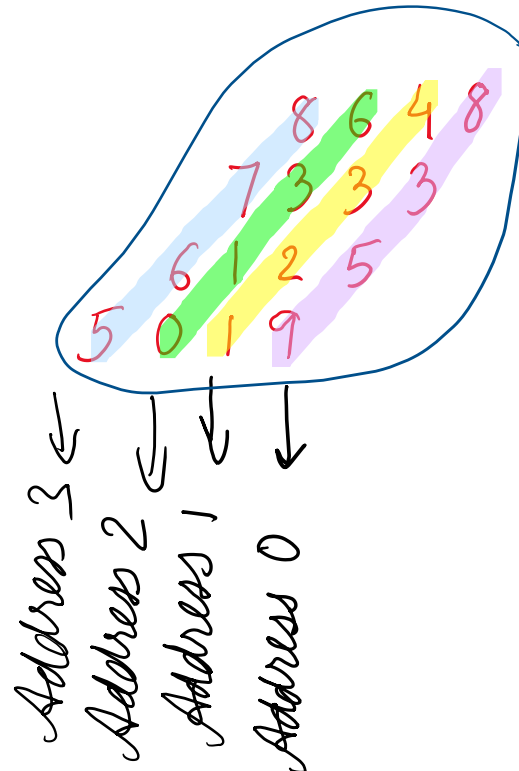


Movement of output matrix

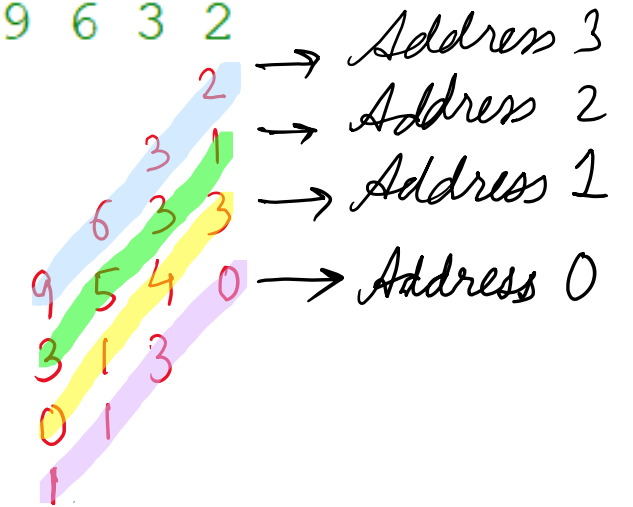
Data layout in RAMs

- Row Major in RAM for matrix A
- Column Major in RAM for matrix B

8	4	6	8
3	3	3	7
5	2	1	6
9	1	0	5



1	1	3	0
0	1	4	3
3	5	3	1
9	6	3	2



Systolic
Matrix
Multiplier

Designing a Systolic MatMul in Verilog

Ports/Interface

Precision

Memory port widths

Address Strides

Validity Mask

Clock counting

PE mesh

PE architecture

Rounding

Memory instances (ext ports)